

猫狗打工队

PetWorkCrew (英文名) • V2.3.0

Source code excerpt (first 30 + last 30 pages)

Scope: Assets_Game own source. Files: 31 / Lines: 11818 / 50 lines per page / Total pages: 237

Generated: 2026-09-22 14:01:18 (local build chain, headless Edge print)

```
// ===== FILE: Assets\_Game\Scripts\Editor\AchievementCheck.cs =====
using System;
using System.Collections.Generic;
using System.IO;
using System.Text;
using Sy.PetWorkCrew.Game;
using UnityEditor;
using UnityEngine;

namespace Sy.PetWorkCrew
{
    /// <summary>
    /// T012 gate (SYS08 honor wall): the eight-achievement unlock chain (requirement
    /// doc section 6) plus the zero-ad U091 declaration honor. Pure C# chain
    /// assertions on PetSim (first wage -> full stars -> postcard collector ->
    /// top-rank cascade GlobalCorp + ZeroAdClear -> fire-open full crew -> event
    /// witness with the legend riding the same evaluation), an ad-watcher twin
    /// proving the zero-ad condition is a real gate (R5 with one ad = no
    /// declaration honor), and a real-view bot session through the honor wall page
    /// (faces, fly beats, modal mutex, evt_zero_ad_clear telemetry write, one-way
    /// law, dual-run fingerprints).
    /// Report: Logs/achievement_report.txt. Exit 0 = all pass.
    /// </summary>
    public static class AchievementCheck
    {
        const float ChunkDt = 300f;    // cadence chunk: one event roll window per chunk

        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;

        static void Assert(bool ok, string label)
        {
            if (ok) { _passed++; Lines.Add("[PASS] " + label); }
            else { _failed++; Lines.Add("[FAIL] " + label); }
        }

        static string Bits(PetSim s)
        {
            var sb = new StringBuilder(PetData.AchievementCount);
            for (int a = 0; a < PetData.AchievementCount; a++)
                sb.Append(s.IsAchievementUnlocked(a) ? '1' : '0');
            return sb.ToString();
        }

        /// <summary>Which achievement ids flipped between two bit vectors ("{5,6}").</summary>
        static string Delta(string before, string after)
        {
            var sb = new StringBuilder("");
            for (int a = 0; a < before.Length && a < after.Length; a++)

```

```

        if (before[a] != after[a]) sb.Append(a).Append(',');
        if (sb.Length > 1) sb.Length--;
        return sb.Append('}').ToString();
    }

    static string BuildBits(bool[] flipped)
    {
        var sb = new StringBuilder(flipped.Length);
        for (int a = 0; a < flipped.Length; a++) sb.Append(flipped[a] ? '1' : '0');
        return sb.ToString();
    }

    static int FirstIdle(PetSim sim)
    {
        for (int p = 0; p < PetData.PetCount; p++)
        {
            if (!sim.IsPetRecruited(p)) continue;
            bool onDuty = false;
            for (int j = 0; j < PetData.JobCount; j++)
                if (sim.JobOccupant(j) == p) { onDuty = true; break; }
            if (!onDuty) return p;
        }
        return -1;
    }

    /// <summary>Growth taps: buy the next pal (awang held out - the fire gift
    /// owns his acquisition path), open the next post, promote while below the
    /// held top gate, place idle pets into empty posts. Deterministic tap order
    /// keeps the chain reproducible across the dual run.</summary>
    static void GrowthTaps(PetSim sim)
    {
        for (int p = 0; p < PetData.PetCount - 1; p++) // awang (index 11) rides the fire gift
            if (!sim.IsPetRecruited(p)) { sim.RecrutPet(p); break; }
        for (int j = 0; j < PetData.JobCount - 1; j++) // fire opens after the top promote
            if (!sim.IsJobUnlocked(j)) { sim.OpenJob(j); break; }
        if (sim.RankIndex < PetData.RankCount - 2) sim.TryPromote();
        for (int j = 0; j < PetData.JobCount - 1; j++)
            if (sim.IsJobUnlocked(j) && sim.JobOccupant(j) < 0)
            {
                int idle = FirstIdle(sim);
                if (idle < 0) break;
                sim.AssignPet(j, idle);
            }
    }

    /// <summary>
    /// Pure-sim honor chain session. watchAd=true burns one rescue ad early so
    /// the twin proves the zero-ad condition is a real gate. The captured faces
    /// feed both the chain asserts and the dual-run determinism fingerprint.
    /// </summary>

```

```

sealed class ChainSession
{
    public string BitsAfterFirstBeat = "";
    public string GrowthDelta = "(none)"; // the only honor that can flip in growth: full stars
    public int StarTop;
    public int CardsAfterSettles;
    public string DeltaPostcardOp = "(none)";
    public string DeltaTop = "";
    public string DeltaFire = "";
    public string DeltaAlbum = "(none)";
    public bool AlbumFilled;
    public string FinalBits = "";
    public int Ads;
    public int UnlockedCount;
    public bool All;
    public int LastAch;
    public int CountAfterOneWay;
    public int RankTop;
    public string Fp = "";

    public string Key()
    {
        return BitsAfterFirstBeat + "|" + GrowthDelta + "|" + StarTop + "|"
            + CardsAfterSettles + "|" + DeltaPostcardOp + "|" + DeltaTop + "|"
            + DeltaFire + "|" + DeltaAlbum + "|" + AlbumFilled + "|" + FinalBits + "|"
            + Ads + "|" + UnlockedCount + "|" + All + "|" + LastAch + "|"
            + CountAfterOneWay + "|" + RankTop + "|" + Fp;
    }
}

static ChainSession PlayChain(bool watchAd)
{
    var sim = new PetSim();
    var s = new ChainSession();
    sim.SeedStarterState();
    if (watchAd) sim.TryActivateRescueBoost(true); // the twin burns exactly one rewarded ad

    // beat 1: the first wage honors the first shift (SYS01 P1 anchor)
    sim.AssignPet(PetData.JobCoffee, PetData.PetOrange);
    sim.Tick(1f);
    s.BitsAfterFirstBeat = Bits(sim);

    // growth: recruits 0..10 (awang rides the fire open), jobs 0..8, promotes
    // held below the top gate, staff every open post. Only the full-stars
    // honor can flip in here (dex capped at 11, fire shut, rank held).
    int guard = 0;
    var growthFlips = new bool[PetData.AchievementCount];
    while ((sim.DexCount < PetData.PetCount - 1 || sim.JobsOpenCount < PetData.JobCount - 1
        || sim.StarLevel < PetData.StarMax) && guard++ < 4000)
    {

```

```

    string b = Bits(sim);
    sim.Tick(ChunkDt);
    GrowthTaps(sim);
    string a = Bits(sim);
    if (b != a)
        for (int f = 0; f < a.Length; f++)
            if (b[f] != a[f]) growthFlips[f] = true;
}
s.GrowthDelta = Delta("00000000", BuildBits(growthFlips));
s.StarTop = sim.StarLevel;

// postcard collector: three away settles pump the journal to the
// collector tier (each settle credits the on-duty pair bars + one
// crew letter; the flip op is captured wherever it lands)
for (int i = 0; i < 3; i++)
{
    string b = Bits(sim);
    sim.SettleOffline(45000.0);
    string a = Bits(sim);
    if (b != a) s.DeltaPostcardOp = Delta(b, a);
}
s.CardsAfterSettles = sim.PostcardsSentTotal;

// the held top promote: R4 -> R5 in one call
string bTop = Bits(sim);
sim.TryPromote();
s.DeltaTop = Delta(bTop, Bits(sim));

// fire open gifts awang -> dex 12 -> full crew
string bFire = Bits(sim);
int fireGuard = 0;
while (sim.Coin < PetData.Jobs[PetData.JobFire].OpenCost && fireGuard++ < 4000) sim.Tick(ChunkDt);
sim.OpenJob(PetData.JobFire);
s.DeltaFire = Delta(bFire, Bits(sim));

// staff the fire post, then ride the cadence until the whole event
// pool is witnessed (E04 needs the fire post on duty)
int idleForFire = FirstIdle(sim);
if (idleForFire >= 0) sim.AssignPet(PetData.JobFire, idleForFire);
int evGuard = 0;
while (!sim.EventAlbumFull && evGuard++ < 4000)
{
    string b = Bits(sim);
    sim.Tick(ChunkDt);
    string a = Bits(sim);
    if (b != a) s.DeltaAlbum = Delta(b, a);
}
s.AlbumFilled = sim.EventAlbumFull;

// one-way law + final faces

```

```

sim.Tick(ChunkDt);
sim.SettleOffline(100.0);
sim.TryPromote();
s.FinalBits = Bits(sim);
s.Ads = sim.AdsWatchedCount;
s.UnlockedCount = sim.AchievementUnlockedCount;
s.All = sim.AllAchievementsUnlocked;
s.LastAch = sim.LastAchievementId;
s.CountAfterOneWay = sim.AchievementUnlockedCount;
s.RankTop = sim.RankIndex;
s.Fp = sim.ComputeFingerprint();
return s;
}

/// <summary>
/// Real-view honor wall session: the bot drives the same chain through the
/// actual play path (Initialize / DriveFrame / taps) - wall faces, unlock
/// fly beats, button count, modal mutex, evt_zero_ad_clear telemetry and
/// the rescue-ad path flipping the ads counter. Big-dt frames keep the
/// whole chain inside a gate session (zero real clock, fully deterministic).
/// </summary>
sealed class ViewSession
{
    public string HonorsFaceBoot = "";
    public bool WallClosedAtBoot;
    public int WallCellsOnOpen;
    public string Cell0LockedFace = "";
    public string Cell6LockedFace = "";
    public bool MutexClosedWall;
    public string HonorsFaceAfterFirstBeat = "";
    public int AchPopAfterFirstBeat;
    public string FlyAfterFirstBeat = "";
    public string Cell0UnlockedFace = "";
    public int AdsAtFull;
    public int ZeroAdWritesAtFull;
    public int UnlockedAtFull;
    public bool AllAtFull;
    public int LastAchAtFull;
    public string Cell7GoldFace = "";
    public string Cell6GoldFace = "";
    public int AdsAfterRescue;
    public bool ZeroAdStillUnlockedAfterRescue;
    public int ZeroAdWritesAfterRescue;
    public int UnlockedAfterOneWay;
    public string WallTitleParity = "";

    public string Key()
    {
        return HonorsFaceBoot + "|" + WallClosedAtBoot + "|" + WallCellsOnOpen + "|"
            + Cell0LockedFace + "|" + Cell6LockedFace + "|" + MutexClosedWall + "|"

```

```

        + HonorsFaceAfterFirstBeat + "|" + AchPopAfterFirstBeat + "|" + FlyAfterFirstBeat + "|"
        + Cell0UnlockedFace + "|" + AdsAtFull + "|" + ZeroAdWritesAtFull + "|"
        + UnlockedAtFull + "|" + AllAtFull + "|" + LastAchAtFull + "|"
        + Cell7GoldFace + "|" + Cell6GoldFace + "|" + AdsAfterRescue + "|"
        + ZeroAdStillUnlockedAfterRescue + "|" + ZeroAdWritesAfterRescue + "|"
        + UnlockedAfterOneWay + "|" + WallTitleParity;
    }
}

static ViewSession PlayView()
{
    var go = new GameObject("BotHonorHost");
    var host = go.AddComponent<PetMainHost>();
    host.Initialize(); // real boot path (Awake is silent in edit mode)
    var s = new ViewSession();

    // boot faces: the wall counts zero honors and stays shut
    s.HonorsFaceBoot = host.HonorsButtonFace;
    s.WallClosedAtBoot = !host.HonorWallOpen;

    // open the wall: 8 cells, locked markers on the goal faces
    host.TapHonors();
    s.WallCellsOnOpen = host.HonorCellFaceCount;
    s.Cell0LockedFace = host.HonorCellFace(PetData.AchFirstWage);
    s.Cell6LockedFace = host.HonorCellFace(PetData.AchZeroAdClear);

    // modal mutex: opening the dex folds the wall (one modal at a time)
    host.TapDex();
    s.MutexClosedWall = !host.HonorWallOpen && host.DexOpen;
    host.TapDex(); // close the dex again
    host.TapHonors(); // reopen the wall

    // first wage honor: dispatch + one frame -> same-frame fly + count flip
    host.TapJob(PetData.JobCoffee);
    host.DriveFrame(1f);
    s.AchPopAfterFirstBeat = host.AchPopCount;
    s.FlyAfterFirstBeat = host.LastAchievementFly;
    s.HonorsFaceAfterFirstBeat = host.HonorsButtonFace;
    s.Cell0UnlockedFace = host.HonorCellFace(PetData.AchFirstWage);
    host.TapHonors(); // close the wall before the growth run

    // full chain through the real play path: taps + big-dt frames +
    // away settles through the view entry (ShowOfflineWelcome)
    int guard = 0;
    int settles = 0;
    while (host.BotAchUnlockedCount < PetData.AchievementCount && guard++ < 6000)
    {
        host.DriveFrame(ChunkDt);
        host.TapRecruit();
        for (int j = 0; j < PetData.JobCount; j++) host.TapJobCell(j);
    }
}

```

```

    if (host.BotRankIndex < PetData.RankCount - 2
        || host.BotAchUnlocked(PetData.AchPostcardCollector))
        host.TapPromote();
    for (int j = 0; j < PetData.JobCount; j++)
    {
        if (!host.BotJobUnlocked(j) || host.BotOccupantOf(j) >= 0) continue;
        if (host.BotFirstIdlePet < 0) break;
        host.TapJob(j);
    }
    // postcard pump: away settles through the view face fill the pair
    // bars + journal to the collector tier (three settles reach 12)
    if (settles < 3
        && host.BotJobsOpenCount >= PetData.JobCount - 1
        && host.OnDutyPets >= PetData.JobCount - 1
        && !host.BotAchUnlocked(PetData.AchPostcardCollector))
    {
        host.ShowOfflineWelcome(45000.0);
        host.TapOfflineClose();
        settles++;
    }
}

s.AdsAtFull = host.BotAdsWatched;
s.ZeroAdWritesAtFull = host.ZeroAdClearWrites;
s.UnlockedAtFull = host.BotAchUnlockedCount;
s.AllAtFull = host.BotAchAll;
s.LastAchAtFull = host.LastAchId;

// the full-gold wall: every cell shows the unlocked marker
host.TapHonors();
s.Cell17GoldFace = host.HonorCellFace(PetData.AchWorkLegend);
s.Cell6GoldFace = host.HonorCellFace(PetData.AchZeroAdClear);

// rescue ad AFTER the wall is full: the ads counter flips (honesty),
// the zero-ad honor never re-locks (one-way law), the telemetry
// write count stays one (the declaration fired once, on the unlock)
host.TapHonors(); // close the wall first
host.TapRescueOffer();
s.AdsAfterRescue = host.BotAdsWatched;
s.ZeroAdStillUnlockedAfterRescue = host.BotAchUnlocked(PetData.AchZeroAdClear);
s.ZeroAdWritesAfterRescue = host.ZeroAdClearWrites;

// one-way law: more play never drops the count
host.DriveFrame(ChunkDt);
s.UnlockedAfterOneWay = host.BotAchUnlockedCount;

s.WallTitleParity = PetMainHost.ComposeHonorWallTitle();

UnityEngine.Object.DestroyImmediate(go); // edit-mode teardown, scene stays untouched
return s;

```

```

    }

    [MenuItem("Sy/G15/AchievementCheck")]
    public static void Run()
    {
        int code;
        try
        {
            // ----- L1/L2 static table laws (PetData mirror, requirement doc section 6) -----
            Assert(PetData.AchievementCount == 8, "L1 achievement table count exact 8 (doc row count)");
            Assert(PetData.SchemaVersion >= 5, "L1 schema version carries the achievement table (v5+ monotonic - T013 slice C
bumped v6)");

            bool idLaw = true;
            var expectedIds = new[]
            {
                "ach_first_wage", "ach_full_crew", "ach_shop_star_full", "ach_postcard_collector",
                "ach_event_witness", "ach_global_corp", "ach_zero_ad_clear", "ach_work_legend",
            };
            for (int a = 0; a < PetData.AchievementCount; a++)
            {
                if (PetData.Achievements[a].Id != expectedIds[a]) idLaw = false;
            }
            Assert(idLaw, "L1 achievement ids mirror the doc table order");
            Assert(PetData.Achievements[PetData.AchZeroAdClear].NameKey == "zero_ad_achievement",
                "L1 zero-ad honor reuses the frozen zero_ad_achievement copy key (U091 declaration)");
            Assert(PetData.PostcardCollectorTarget == 12,
                "L1 postcard collector tier target 12 (a dozen cards)");
            bool nameKeys = true;
            for (int a = 0; a < PetData.AchievementCount; a++)
            {
                if (!CopyTable.HasKey(PetData.Achievements[a].NameKey)) nameKeys = false;
            }
            Assert(nameKeys, "L2 every achievement NameKey resolves in the copy table");

            // ----- L3..L9: the clean zero-ad chain (pure sim) -----
            var clean = PlayChain(false);

            Assert(clean.BitsAfterFirstBeat == "10000000",
                "L3 first wage honors the first beat, nothing else (bits '" + clean.BitsAfterFirstBeat + "')");
            Assert(clean.GrowthDelta == "{2}" && clean.StarTop >= PetData.StarMax,
                "L4 full-stars honor flips alone in growth once the shop star tops out (delta '"
                + clean.GrowthDelta + "', star " + clean.StarTop + "')");
            Assert(clean.DeltaPostcardOp == "{3}",
                "L5 postcard collector flips alone on the journal op (delta '" + clean.DeltaPostcardOp
                + "', cards " + clean.CardsAfterSettles + "')");
            Assert(clean.DeltaTop == "{5,6}",
                "L6 top promote cascades GlobalCorp + ZeroAdClear in one call (delta '" + clean.DeltaTop + "')");
            Assert(clean.DeltaFire == "{1}",
                "L7 fire open gifts the 12th pal - full crew flips alone (delta '" + clean.DeltaFire + "')");
            Assert(clean.AlbumFilled && clean.DeltaAlbum == "{4,7}",
                "L8 event witness completes the pool and the legend rides the same evaluation (delta '"
                + clean.DeltaAlbum + "')");
            Assert(clean.FinalBits == "11111111" && clean.Ads == 0 && clean.UnlockedCount == 8
                && clean.LastAch == PetData.AchWorkLegend,
                "L8b clean chain ends: all bits set, zero ads, legend last (ads=" + clean.Ads

```

```

    + ", n=" + clean.UnlockedCount + ", last=" + clean.LastAch + ")");
    Assert(clean.CountAfterOneWay == 8,
        "L9 one-way law: extra play never drops or re-locks a honor (n=" + clean.CountAfterOneWay + ")");

    // ----- L10: the ad-watcher twin - the zero-ad condition is a real gate -----
    var twin = PlayChain(true);
    Assert(twin.Ads == 1 && twin.FinalBits == "11111100"
        && twin.UnlockedCount == 6 && !twin.All,
        "L10 one watched ad disqualifies the declaration honor and the legend (ads=" + twin.Ads
        + ", bits '" + twin.FinalBits + "', n=" + twin.UnlockedCount + ")");
    Assert(twin.DeltaTop == "{5}",
        "L10b the twin's top promote unlocks the rank honor alone (delta '" + twin.DeltaTop + "')");

    // ----- L11: dual-run determinism (clean chain fingerprint) -----
    var rerun = PlayChain(false);
    Assert(clean.Key() == rerun.Key(),
        "L11 dual-run chain determinism byte-identical (fingerprint face)");

    // ----- V1..V9: the real-view honor wall session -----
    var v1 = PlayView();
    Assert(v1.HonorsFaceBoot == "Honor Wall 0/8" && v1.WallClosedAtBoot,
        "V1 boot honors face 'Honor Wall 0/8', wall shut: '" + v1.HonorsFaceBoot + "'");
    Assert(v1.WallCellsOnOpen == 8
        && v1.Cell0LockedFace == "First Paycheck! - Locked..."
        && v1.Cell6LockedFace == "Zero-Ad Tycoon - Locked...",
        "V2 wall opens with 8 composer-exact locked cells ('" + v1.Cell0LockedFace
        + "' / '" + v1.Cell6LockedFace + "')");
    Assert(v1.MutexClosedWall, "V3 modal mutex: opening the dex folds the wall");
    Assert(v1.AchPopAfterFirstBeat == 1
        && v1.FlyAfterFirstBeat == "First Paycheck! - Unlocked!"
        && v1.HonorsFaceAfterFirstBeat == "Honor Wall 1/8"
        && v1.Cell0UnlockedFace == "First Paycheck! - Unlocked!",
        "V4 first wage unlock: same-frame fly + count flip + gold cell ('"
        + v1.FlyAfterFirstBeat + "', '" + v1.HonorsFaceAfterFirstBeat + "')");
    Assert(v1.AdsAtFull == 0 && v1.AllAtFull && v1.UnlockedAtFull == 8
        && v1.LastAchAtFull == PetData.AchWorkLegend && v1.ZeroAdWritesAtFull == 1,
        "V5 view chain reaches the full wall: all 8, zero ads, one evt_zero_ad_clear write (ads="
        + v1.AdsAtFull + ", writes=" + v1.ZeroAdWritesAtFull + ")");
    Assert(v1.Cell7GoldFace == "Work Legend! - Unlocked!"
        && v1.Cell6GoldFace == "Zero-Ad Tycoon - Unlocked!",
        "V6 full-gold wall faces ('" + v1.Cell7GoldFace + "' / '" + v1.Cell6GoldFace + "')");
    Assert(v1.AdsAfterRescue == 1 && v1.ZeroAdStillUnlockedAfterRescue
        && v1.ZeroAdWritesAfterRescue == 1,
        "V7 rescue ad after the wall: counter honest (ads=" + v1.AdsAfterRescue
        + "), zero-ad honor never re-locks, telemetry write stays one");
    Assert(v1.UnlockedAfterOneWay == 8, "V8 one-way law through the view (n=" + v1.UnlockedAfterOneWay + ")");
    Assert(v1.WallTitleParity == CopyTable.Get(CopyTable.LangEn, "honor_wall"),
        "V9 wall title routes the copy table (zero hardcode)");

    // ----- V10: dual-run view determinism -----

```

```

        var v2 = PlayView();
        Assert(v1.Key() == v2.Key(), "V10 dual-run view determinism byte-identical");

        Lines.Add("ALL_GREEN passed=" + _passed + " failed=" + _failed);
        code = _failed == 0 ? 0 : 1;
    }
    catch (System.Exception e)
    {
        Lines.Add("[FAIL] exception: " + e.Message);
        code = 1;
    }

    string report = Path.GetFullPath(Path.Combine(Application.dataPath, "..",
        "Logs", "achievement_report.txt"));
    Directory.CreateDirectory(Path.GetDirectoryName(report));
    File.WriteAllText(report, string.Join(Environment.NewLine, Lines.ToArray()) + Environment.NewLine,
        new UTF8Encoding(false));
    Console.WriteLine("AchievementCheck passed=" + _passed + " failed=" + _failed);
    EditorApplication.Exit(code);
}
}

// ===== FILE: Assets\\Game\\Scripts\\Editor\\AdCapBoostCheck.cs =====
using System.Collections.Generic;
using System.IO;
using System.Text;
using Sy.PetWorkCrew.Game;
using Sy.Platform;
using Sy.Telemetry;
using UnityEditor;
using UnityEngine;

namespace Sy.PetWorkCrew
{
    /// <summary>
    /// T007 slice A: IAA triple AdCap gate (batchmode entry, pure C#, zero scene,
    /// zero RNG): Tuanjie.exe -batchmode -quit -executeMethod Sy.PetWorkCrew.AdCapBoostCheck.Run -logFile &lt;abs>
    /// Asserts the numeric pack section 7 laws: the three seats (rescue 5/day,
    /// speed 8/day, offline double 1/day) own g15_-isolated categories, the AdCap
    /// pre-check + consume caps hold (category caps first, then the 12/day global
    /// hard cap squeezing the 14-seat sum), the sim grant ops are ad-certified
    /// (unconsumed view = zero-state-change refusal, empty job never wastes the
    /// ad), the wage windows multiply beats by exactly x2 (shop-wide rescue,
    /// single-job speed, stacking never compounds beyond the x2 red line,
    /// 30-minute expiry returns to base), the offline double claim gates on the
    /// consumed ad with the slot staying honest, the evt_ad funnel telemetry
    /// (offer/show/complete/landed x 3 seats) rides the unified schema clean,
    /// and identical (dt, op) streams stay byte-identical (zero-RNG law).
    /// Exit codes: 0=all assertions pass 1=any fail or exception.
    /// </summary>

```

```

public static class AdCapBoostCheck
{
    static readonly List<string> Lines = new List<string>();
    static int _passed;
    static int _failed;

    static void Assert(bool ok, string label)
    {
        if (ok) { _passed++; Lines.Add("[PASS] " + label); }
        else { _failed++; Lines.Add("[FAIL] " + label); }
    }

    /// <summary>
    /// Deterministic two-job shop (EventEngineCheck scenario shape): boot
    /// dispatch, recruit snowball with earned wages, grind to star 1, promote
    /// to rank 1, open + staff the courier job. Beat deltas append to
    /// <paramref name="coffee"/> / <paramref name="courier"/> when non-null.
    /// </summary>
    static PetSim BuildShop(List<double> coffee, List<double> courier)
    {
        var sim = new PetSim();
        sim.SeedStarterState();
        sim.AssignPet(PetData.JobCoffee, PetData.PetOrange);
        for (int i = 0; i < 10; i++) sim.Tick(1f); // wages cover the 200 recruit cost
        sim.RecrutPet(1); // pet_snowball
        int guard = 0;
        while (sim.StarLevel < 1 && guard++ < 6000) sim.Tick(1f);
        sim.TryPromote();
        sim.OpenJob(PetData.JobCourier);
        sim.AssignPet(PetData.JobCourier, 1);
        for (int i = 0; i < 5; i++) sim.Tick(1f); // settle into steady beats
        if (coffee != null || courier != null)
            sim.OnCoinBeat += (t, delta, jobId) =>
            {
                if (coffee != null && jobId == PetData.JobCoffee) coffee.Add(delta);
                if (courier != null && jobId == PetData.JobCourier) courier.Add(delta);
            };
        return sim;
    }

    /// <summary>Drives n one-second ticks; returns the last captured deltas.</summary>
    static double[] LastBeats(PetSim sim, int n)
    {
        for (int i = 0; i < n; i++) sim.Tick(1f);
        return new double[0];
    }

    /// <summary>
    /// Fixed (dt, op) script with boost ops for the determinism law: rescue +
    /// speed activations, an offline settle with the ad-gated double claim,

```

```

/// and a long walk expiring both windows.
/// </summary>
static string BoostScriptFingerprint()
{
    var sim = new PetSim();
    sim.SeedStarterState();
    sim.AssignPet(PetData.JobCoffee, PetData.PetOrange);
    for (int i = 0; i < 600; i++) sim.Tick(1f);
    sim.TryActivateRescueBoost(true);
    for (int i = 0; i < 30; i++) sim.Tick(1f);
    sim.TryActivateSpeedBoost(PetData.JobCoffee, true);
    for (int i = 0; i < 30; i++) sim.Tick(1f);
    sim.SettleOffline(120.0);
    sim.TryClaimOfflineDouble(true);
    for (int i = 0; i < 1750; i++) sim.Tick(1f);
    return sim.ComputeFingerprint();
}

static bool FunnelHas(System.Collections.Generic.IReadOnlyList<string> f, string entry)
{
    for (int i = 0; i < f.Count; i++) if (f[i] == entry) return true;
    return false;
}

static bool FunnelTailIs(System.Collections.Generic.IReadOnlyList<string> f,
    string a, string b, string c)
{
    return f.Count >= 3 && f[f.Count - 3] == a && f[f.Count - 2] == b && f[f.Count - 1] == c;
}

/// <summary>
/// T007 slice B View section (BotOfflineHost pattern): boots the REAL
/// PetMainHost and replays the three offer seats' play path - the rescue
/// HUD seat, the per-job speed seats and the offline x2 re-presentation
/// chip (pack section 7 "1-5/200" law). Expected values mirror the same
/// twin-sim discipline (X491 lesson: snapshot-first, byte-exact receipts).
/// </summary>
static void RunBoostViewSection()
{
    var go = new GameObject("BotBoostHost");
    var host = go.AddComponent<PetMainHost>();
    host.Initialize();
    host.DriveFrame(0.016f);

    // same-base twin: mirrors every host op for the byte-exact ledger compares
    var twin = new PetSim();
    twin.SeedStarterState();

    // V1: dispatch the starter cat; both callable seats render + offer writes fire
    host.TapJob(PetData.JobCoffee);

```

```

host.DriveFrame(0.5f);
twin.AssignPet(PetData.JobCoffee, PetData.PetOrange);
twin.Tick(0.5f);
var fn = host.BotFunnel;
bool v1 = host.RescueOfferFace == PetMainHost.ComposeRescueOfferFace(false, 0.0, true)
    && host.SpeedCellFace(PetData.JobCoffee) == PetMainHost.ComposeSpeedCellFace(false)
    && FunnelHas(fn, "g15_rescue:offer") && FunnelHas(fn, "g15_speed:offer");
Assert(v1, "V1 boot seats: rescue + speed offers render callable faces, evt_ad offer writes fired (per visible epoch)");

// V2: rescue grant through the full chain: pre-check -> view -> consume -> grant
host.TapRescueOffer();
bool v2 = host.BotRescueWindowOpen
    && host.RescueOfferFace.StartsWith("x2 on!")
    && host.LastBoostFly == PetMainHost.ComposeBoostFly(PetSim.BoostKindRescue, -1)
    && host.BoostPopCount == 1
    && FunnelTailIs(host.BotFunnel, "g15_rescue:show", "g15_rescue:complete", "g15_rescue:landed");
Assert(v2, "V2 rescue grant: window opens, fly + AU06 pop beat + offer>show>complete>landed chain");

// V3: in-window duplicate tap = pre-offer refusal BEFORE the ad call
int writesAtDup = host.BotFunnel.Count;
double coinAtDup = host.CoinFace;
host.TapRescueOffer();
bool v3 = host.LastBoostRefusal == "Already x2 on!"
    && host.BotFunnel.Count == writesAtDup
    && host.CoinFace == coinAtDup && host.BotRescueWindowOpen;
Assert(v3, "V3 in-window duplicate rescue tap: comedy refusal before the ad (zero waste), zero change");

// V4: speed grant on the occupied coffee seat
host.TapSpeedOffer(PetData.JobCoffee);
bool v4 = host.BotSpeedBoostJob == PetData.JobCoffee
    && host.SpeedCellFace(PetData.JobCoffee) == PetMainHost.ComposeSpeedCellFace(true)
    && host.LastBoostFly == PetMainHost.ComposeBoostFly(PetSim.BoostKindSpeed, PetData.JobCoffee)
    && host.BoostPopCount == 2
    && FunnelTailIs(host.BotFunnel, "g15_speed:show", "g15_speed:complete", "g15_speed:landed");
Assert(v4, "V4 speed grant: single-job window opens, seat flips live face, full funnel chain");

// V5: live-window duplicate speed tap mirrors the sim's AlreadyActive
int writesAtSeat = host.BotFunnel.Count;
double coinAtSeat = host.CoinFace;
host.TapSpeedOffer(PetData.JobCoffee);
bool v5 = host.LastBoostRefusal == "Already x2 on!"
    && host.BotFunnel.Count == writesAtSeat
    && host.CoinFace == coinAtSeat && host.BotSpeedBoostJob == PetData.JobCoffee;
Assert(v5, "V5 live-window duplicate speed tap: refusal mirrors AlreadyActive, zero change");

// V6: boosted wage beats mirror the same-base twin byte-exact (x2, never x4)
twin.TryActivateRescueBoost(true);
twin.TryActivateSpeedBoost(PetData.JobCoffee, true);
for (int i = 0; i < 5; i++) { host.DriveFrame(1f); twin.Tick(1f); }
Assert(host.CoinFace == twin.Coin,

```

```

    "V6 boosted wage ledger mirror: rescue+speed stacking beats match the twin byte-exact (x2 cap)");

// V7: offline settle -> close-without-claim arms the 1-5/200 chip window
host.ShowOfflineWelcome(3600.0);
var rt = twin.SettleOffline(3600.0);
bool v7a = host.OfflinePageOpen && FunnelHas(host.BotFunnel, "g15_double:offer");
Assert(v7a, "V7a settle opens the receipt page and fires the double offer write");
host.TapOfflineClose();
host.DriveFrame(0.5f); // blank gap inside the 1-5s anchor
bool gapBlank = host.DoubleChipFace == "";
host.DriveFrame(1.0f); // elapsed 1.5s: presented inside the anchor
bool presented = host.DoubleChipFace == "x2 wages!";
host.DriveFrame(198.5f); // elapsed 200s: still the loud window
bool loudWindow = host.DoubleChipFace == "x2 wages!";
host.DriveFrame(1.5f); // elapsed 201.5s: quiet badge, still tappable
bool quietBadge = host.DoubleChipFace == "x2?";
Assert(gapBlank && presented && loudWindow && quietBadge,
    "V7b chip presentation window (1-5/200): blank gap -> x2 wages! at 1.5s -> loud through 200s -> x2? badge");

double coinAtReopen = host.CoinFace;
host.TapDoubleChip();
bool v7c = host.OfflinePageOpen
    && host.OfflineDoubleFace == PetMainHost.ComposeOfflineDoubleFace(true)
    && host.DoubleChipFace == ""; // the page IS the offer face while open
Assert(v7c, "V7c chip tap reopens the receipt page (no re-settle), chip folds");

host.TapOfflineDouble();
bool v7d = host.CoinFace == coinAtReopen + rt.Coin
    && host.LastOfflineDoubleFly == PetMainHost.ComposeOfflineDoubleFly(rt.Coin)
    && host.DoubleChimeCount == 1
    && FunnelTailIs(host.BotFunnel, "g15_double:show", "g15_double:complete", "g15_double:landed")
    && !host.BotDoublePending && host.DoubleChipFace == "";
Assert(v7d, "V7d double claim through the full chain: +base bonus byte-exact, AU09 chime beat, chip clears");

double coinAtSecond = host.CoinFace;
int writesAtSecond = host.BotFunnel.Count;
host.TapOfflineDouble();
Assert(host.CoinFace == coinAtSecond && host.BotFunnel.Count == writesAtSecond,
    "V7e single-shot second claim: silent no-op, zero writes (slot law before budget law)");

// V8: burn the remaining rescue grants through the REAL seat, each window
// walked to expiry; the 6th tap answers the honest cap face with zero
// change and zero funnel writes (pre-offer gate = no dead ad calls)
for (int i = 0; i < 4; i++)
{
    host.DriveFrame(1805f); // walk past the 30-minute expiry
    host.TapRescueOffer();
}
host.DriveFrame(1805f); // the 5th grant's window expires too
bool capFace = host.RescueOfferFace == PetMainHost.ComposeRescueOfferFace(false, 0.0, false);

```

```

    int writesAtCap = host.BotFunnel.Count;
    double coinAtCap = host.CoinFace;
    host.TapRescueOffer();
    bool v8 = capFace
        && host.LastBoostRefusal == "Cap hit! Tomorrow!"
        && host.BotFunnel.Count == writesAtCap
        && host.CoinFace == coinAtCap && !host.BotRescueWindowOpen;
    Assert(v8, "V8 rescue cap 5/day burned through the real seat: 6th tap = honest cap refusal, zero change, zero writes");
}

sealed class CaptureSink : TelemetryService.ISink
{
    public int Count;
    public void Write(string evtId, Dictionary<string, string> fields) { Count++; }
}

public static void Run()
{
    int code;
    try
    {
        var trace = new StringBuilder();
        const string day = "20260922";

        // ---- law 1: pack section 7 category table mirror ----
        Assert(AdCapPolicy.CategoryCap(AdCategories.G15Rescue) == 5, "rescue seat cap = 5/day (pack section 7 table)");
        Assert(AdCapPolicy.CategoryCap(AdCategories.G15Speed) == 8, "speed seat cap = 8/day (pack section 7 table)");
        Assert(AdCapPolicy.CategoryCap(AdCategories.G15Double) == 1, "offline double seat cap = 1/day (pack section 7
table)");

        Assert(AdCategories.CategoryOfPoint("g15_rescue") == AdCategories.G15Rescue
            && AdCategories.CategoryOfPoint("g15_speed") == AdCategories.G15Speed
            && AdCategories.CategoryOfPoint("g15_double") == AdCategories.G15Double,
            "three seats map to their own g15-isolated categories (no cross-game budget draw)");

        // ---- law 2: AdCap pre-check + consume caps (SYS10 face) ----
        var fresh = new AdCapPolicy();
        Assert(fresh.HasCategoryBudget(day, AdCategories.G15Rescue)
            && fresh.HasCategoryBudget(day, AdCategories.G15Speed)
            && fresh.HasCategoryBudget(day, AdCategories.G15Double),
            "fresh day: all three seats pass the HasCategoryBudget pre-check");

        var rescue = new AdCapPolicy();
        bool rescueAll = true;
        for (int i = 0; i < 5; i++) rescueAll &= rescue.TryConsumeRewarded(day, AdCategories.G15Rescue);
        Assert(rescueAll && !rescue.HasCategoryBudget(day, AdCategories.G15Rescue)
            && !rescue.TryConsumeRewarded(day, AdCategories.G15Rescue),
            "rescue cap 5/day: five views land, the sixth refuses at the pre-offer gate");

        var speed = new AdCapPolicy();
        bool speedAll = true;
        for (int i = 0; i < 8; i++) speedAll &= speed.TryConsumeRewarded(day, AdCategories.G15Speed);
    }
}

```

```

Assert(speedAll && !speed.HasCategoryBudget(day, AdCategories.G15Speed)
    && !speed.TryConsumeRewarded(day, AdCategories.G15Speed),
    "speed cap 8/day: eight views land, the ninth refuses at the pre-offer gate");

var dbl = new AdCapPolicy();
Assert(dbl.TryConsumeRewarded(day, AdCategories.G15Double)
    && !dbl.TryConsumeRewarded(day, AdCategories.G15Double)
    && !dbl.HasCategoryBudget(day, AdCategories.G15Double),
    "offline double cap 1/day: the pack's once-a-day offer law holds");

var squeeze = new AdCapPolicy();
bool first12 = true;
for (int i = 0; i < 5; i++) first12 &= squeeze.TryConsumeRewarded(day, AdCategories.G15Rescue);
for (int i = 0; i < 7; i++) first12 &= squeeze.TryConsumeRewarded(day, AdCategories.G15Speed);
Assert(first12 && squeeze.UsedRewardedToday(day) == 12
    && squeeze.HasCategoryBudget(day, AdCategories.G15Speed) // category still has 1 left
    && !squeeze.TryConsumeRewarded(day, AdCategories.G15Speed), // ...but the global cap binds
    "global 12/day hard cap squeezes the 14-seat sum (13th view refuses while category budget remains)");

// ---- law 3: sim grant ops are ad-certified (refusal family = zero change) ----
var guardSim = BuildShop(null, null);
string fp0 = guardSim.ComputeFingerprint();
bool g1 = guardSim.TryActivateRescueBoost(false) == PetSim.BoostResult.AdNotConsumed;
bool g2 = guardSim.TryActivateSpeedBoost(PetData.JobCoffee, false) == PetSim.BoostResult.AdNotConsumed;
bool g3 = guardSim.TryActivateSpeedBoost(99, true) == PetSim.BoostResult.JobLocked;
Assert(g1 && g2 && g3 && guardSim.ComputeFingerprint() == fp0,
    "ad-gate refusals (unconsumed view / invalid job) = zero state change");

guardSim.AssignPet(PetData.JobCourier, PetData.PetOrange); // coffee left empty (comedy beat, no penalty)
string fp1 = guardSim.ComputeFingerprint();
bool g4 = guardSim.TryActivateSpeedBoost(PetData.JobCoffee, true) == PetSim.BoostResult.JobEmpty;
Assert(g4 && guardSim.ComputeFingerprint() == fp1,
    "empty-job speed boost refused - the ad can never be wasted on a job that produces nothing");

// full SYS10 chain on a minimal boot shop: pre-check -> consume -> grant
var chain = new PetSim();
chain.SeedStarterState();
chain.AssignPet(PetData.JobCoffee, PetData.PetOrange);
var chainCap = new AdCapPolicy();
Assert(chainCap.HasCategoryBudget(day, AdCategories.G15Speed)
    && chainCap.TryConsumeRewarded(day, AdCategories.G15Speed)
    && chain.TryActivateSpeedBoost(PetData.JobCoffee, true) == PetSim.BoostResult.Ok,
    "full SYS10 chain: pre-check -> consume -> ad-certified grant lands");

// ---- law 4: rescue window = shop-wide x2, event probe, duplicate refusal ----
var coffeeBeats = new List<double>();
var courierBeats = new List<double>();
var rescueSim = BuildShop(coffeeBeats, courierBeats);
for (int i = 0; i < 3; i++) rescueSim.Tick(1f); // base beats
double coffeeBase = coffeeBeats[coffeeBeats.Count - 1];

```

```

double courierBase = courierBeats[courierBeats.Count - 1];
Assert(coffeeBase > 0.0 && courierBase > 0.0, "two-job shop earning base wages on both seats");

int firedKind = -1; int firedJob = -2; // declared before the subscription (CS0844 law)
rescueSim.OnAdBoostActivated += (kind, job, t) => { firedKind = kind; firedJob = job; };
bool rescueOk = rescueSim.TryActivateRescueBoost(true) == PetSim.BoostResult.Ok;
Assert(rescueOk && firedKind == PetSim.BoostKindRescue && firedJob == -1 && rescueSim.RescueBoostActive
    && rescueSim.RescueBoostRemainSec <= PetSim.AdBoostDurSec,
    "rescue activation fires the shop-wide boost event (View/telemetry hook, window open)");

string fpR = rescueSim.ComputeFingerprint();
bool dupRescue = rescueSim.TryActivateRescueBoost(true) == PetSim.BoostResult.AlreadyActive;
Assert(dupRescue && rescueSim.ComputeFingerprint() == fpR,
    "in-window duplicate rescue refused (pre-offer gate's honest data face)");

int c0 = coffeeBeats.Count, k0 = courierBeats.Count;
for (int i = 0; i < 3; i++) rescueSim.Tick(1f); // boosted beats
bool shopWide = true;
for (int i = c0; i < coffeeBeats.Count; i++) shopWide &= coffeeBeats[i] == 2.0 * coffeeBase;
for (int i = k0; i < courierBeats.Count; i++) shopWide &= courierBeats[i] == 2.0 * courierBase;
Assert(shopWide, "rescue window doubles EVERY occupied job's beat exactly (x2 shop-wide)");

LastBeats(rescueSim, 1805); // walk past the 30-minute expiry
bool expired = coffeeBeats[coffeeBeats.Count - 1] == coffeeBase
    && courierBeats[courierBeats.Count - 1] == courierBase
    && !rescueSim.RescueBoostActive;
Assert(expired, "rescue window expires after 30 minutes - beats return to base (no permanent gain)");

// ---- law 5: speed window = single job, stacking caps at x2, single seat ----
var sc = new List<double>();
var kc = new List<double>();
var speedSim = BuildShop(sc, kc);
for (int i = 0; i < 3; i++) speedSim.Tick(1f); // base beats
double sCoffeeBase = sc[sc.Count - 1];
double sCourierBase = kc[kc.Count - 1];
bool speedOk = speedSim.TryActivateSpeedBoost(PetData.JobCoffee, true) == PetSim.BoostResult.Ok;
int s0 = sc.Count, k1 = kc.Count;
for (int i = 0; i < 3; i++) speedSim.Tick(1f);
bool singleJob = speedOk && speedSim.SpeedBoostJob == PetData.JobCoffee;
for (int i = s0; i < sc.Count; i++) singleJob &= sc[i] == 2.0 * sCoffeeBase;
for (int i = k1; i < kc.Count; i++) singleJob &= kc[i] == sCourierBase;
Assert(singleJob, "speed window doubles ONLY the chosen job (other jobs keep base beats)");

string fpS = speedSim.ComputeFingerprint();
bool reAim = speedSim.TryActivateSpeedBoost(PetData.JobCourier, true) == PetSim.BoostResult.AlreadyActive;
Assert(reAim && speedSim.ComputeFingerprint() == fpS,
    "live speed window is single-seat: re-aiming at another job refuses (zero change)");

speedSim.TryActivateRescueBoost(true); // stacking window: rescue on top of speed
int s1 = sc.Count, k2 = kc.Count;

```

```

for (int i = 0; i < 3; i++) speedSim.Tick(1f);
bool stackCapped = true;
for (int i = s1; i < sc.Count; i++) stackCapped &= sc[i] == 2.0 * sCoffeeBase; // never x4
for (int i = k2; i < kc.Count; i++) stackCapped &= kc[i] == 2.0 * sCourierBase; // rescue covers it
Assert(stackCapped, "stacking law: rescue + speed on one job caps at x2 exactly (never x4)");

LastBeats(speedSim, 1805);
bool speedExpired = sc[sc.Count - 1] == sCoffeeBase && kc[kc.Count - 1] == sCourierBase
    && speedSim.SpeedBoostJob == -1;
Assert(speedExpired, "speed window expires after 30 minutes - beats return to base");

// ---- law 6: offline double claim gates on the consumed ad ----
var off = new PetSim();
off.SeedStarterState();
off.AssignPet(PetData.JobCoffee, PetData.PetOrange);
for (int i = 0; i < 30; i++) off.Tick(1f);
var settle = off.SettleOffline(3600.0);
Assert(settle.Coin > 0.0 && off.HasDoubleOfferPending,
    "settle credits offline wages and raises the claimable x2 offer slot");

string fpOff = off.ComputeFingerprint();
double coinBefore = off.Coin;
bool noAd = !off.TryClaimOfflineDouble(false);
Assert(noAd && off.HasDoubleOfferPending && off.Coin == coinBefore && off.ComputeFingerprint() == fpOff,
    "claim without a consumed ad refused - zero change, the base is never held hostage, slot stays claimable");

bool withAd = off.TryClaimOfflineDouble(true);
Assert(withAd && off.Coin == coinBefore + settle.Coin * (PetSim.OfflineDoubleMult - 1.0)
    && !off.HasDoubleOfferPending && !off.TryClaimOfflineDouble(true),
    "claim with the consumed ad grants exactly one base more (x2 total) and the slot single-shots");

// ---- law 7: evt_ad funnel telemetry rides the unified schema clean ----
var svc = new TelemetryService();
var def = new EventDef { Id = EventIds.Ad };
def.Fields.Add(new FieldDef { Name = "state", Type = "string", Required = true });
def.Fields.Add(new FieldDef { Name = "pointId", Type = "string", Required = true });
def.Fields.Add(new FieldDef { Name = "jobId", Type = "int" });
def.Fields.Add(new FieldDef { Name = "dur", Type = "number" });
def.Fields.Add(new FieldDef { Name = "span", Type = "number" });
def.Fields.Add(new FieldDef { Name = "gain", Type = "number" });
svc.RegisterEvent(def);
var sink = new CaptureSink();
svc.AddSink(sink);
string[] states = { "offer", "show", "complete", "landed" };
foreach (var state in states)
{
    svc.Track(EventIds.Ad, new Dictionary<string, string>
        { { "state", state }, { "pointId", "g15_rescue" }, { "jobId", "0" }, { "dur", "1800" } });
    svc.Track(EventIds.Ad, new Dictionary<string, string>
        { { "state", state }, { "pointId", "g15_speed" }, { "jobId", "0" }, { "dur", "1800" } });
}

```

```

        svc.Track(EventIds.Ad, new Dictionary<string, string>
            { { "state", state }, { "pointId", "g15_double" }, { "span", "3600" }, { "gain", "1" } });
    }
    Assert(sink.Count == 12 && svc.LastWarnings.Count == 0,
        "evt_ad funnel telemetry: 3 seats x offer/show/complete/landed pass the schema clean (12 writes, zero
warnings)");

    // ---- law 8: determinism + red-line constants ----
    Assert(BoostScriptFingerprint() == BoostScriptFingerprint(),
        "deterministic fingerprint with the boost face (identical dt+op streams, byte-identical)");
    Assert(PetSim.AdBoostAmp == 2.0 && PetSim.OfflineDoubleMult == 2.0 && PetSim.AdBoostDurSec == 1800.0,
        "red line: every ad gain = exactly x2, every window = 30 minutes (R73/U091 law)");

    // ---- law 9 (T007 slice B): the three offer faces through the real host ----
    RunBoostViewSection();

    trace.Append("caps=5/8/1 globalSqueeze=12 rescueBeats=").Append(coffeeBeats.Count)
        .Append(" speedStackCapped=true evtAdWrites=").Append(sink.Count).AppendLine();

    // report (first line = ALL_GREEN family convention for wave scripts)
    string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
    var logsDir = Path.Combine(projectRoot, "Logs");
    if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);

    var sb = new StringBuilder();
    sb.AppendLine("ALL_GREEN passed=" + _passed + " failed=" + _failed);
    sb.Append(trace);
    foreach (var line in Lines) sb.AppendLine(line);
    File.WriteAllText(Path.Combine(logsDir, "adcapboost_report.txt"), sb.ToString(), new UTF8Encoding(false));
    foreach (var line in Lines) Debug.Log(line);
    Debug.Log("[AdCapBoostCheck] " + (_failed == 0 ? "ALL_GREEN" : "FAILED") + " passed=" + _passed + " failed=" +
_failed);

    code = _failed == 0 ? 0 : 1;
}
catch (System.Exception e)
{
    Debug.LogError("[AdCapBoostCheck] exception: " + e);
    code = 1;
}
}
EditorApplication.Exit(code);
}
}
}
}
}
// ===== FILE: Assets\_Game\Scripts\Editor\BootSequenceCheck.cs =====
using System.IO;
using System.Text;
using UnityEditor;
using UnityEditor.SceneManagement;
using UnityEngine;

namespace Sy.PetWorkCrew

```

```

{
    /// <summary>
    /// Seven-step boot dry-run (batchmode entry):
    /// Tuanjie.exe -batchmode -quit -executeMethod Sy.PetWorkCrew.BootSequenceCheck.Run -logFile <abs>
    /// Exit codes: 0=all steps pass and well formed 1=fail/exception 3=scenes missing (run ProjectSetup first)
    /// </summary>
    public static class BootSequenceCheck
    {
        public static void Run()
        {
            int code;
            try
            {
                string mainAsset = "Assets/_Game/Scenes/S15_Main.unity";
                string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
                if (!File.Exists(Path.Combine(projectRoot, mainAsset)))
                {
                    Debug.LogError("[BootSequenceCheck] main scene missing - run ProjectSetup first");
                    EditorApplication.Exit(3);
                    return;
                }

                Boot.BootServices.RegisterAll();
                bool switched = false;
                var sequence = new Boot.BootSequence(delegate
                {
                    EditorSceneManager.OpenScene(mainAsset, OpenSceneMode.Single);
                    switched = true;
                    return true;
                });
                bool allOk = sequence.RunAll();

                var sb = new StringBuilder();
                foreach (var line in sequence.Lines) sb.AppendLine(line);
                var logsDir = Path.Combine(projectRoot, "Logs");
                if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
                File.WriteAllText(Path.Combine(logsDir, "boot_sequence_report.txt"), sb.ToString(), new UTF8Encoding(false));

                bool wellFormed = sequence.Lines.Count == Boot.BootSequence.TotalSteps && switched;
                for (int i = 0; i < sequence.Lines.Count; i++)
                {
                    Debug.Log(sequence.Lines[i]);
                    if (!sequence.Lines[i].StartsWith("[BOOT] " + (i + 1) + "/" + Boot.BootSequence.TotalSteps + " "))
                        wellFormed = false;
                }

                code = allOk && wellFormed ? 0 : 1;
            }
            catch (System.Exception e)
            {

```

```

        Debug.LogError("[BootSequenceCheck] exception: " + e);
        code = 1;
    }
    EditorApplication.Exit(code);
}
}
}
}
// ===== FILE: Assets\_Game\Scripts\Editor\BotBarePlayCheck.cs =====
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Text;
using Sy.PetWorkCrew.Game;
using UnityEditor;
using UnityEngine;

namespace Sy.PetWorkCrew
{
    /// <summary>
    /// T002 slice B-1c bot bare-play gate (batchmode entry, T008 timestamp-capture
    /// law family): a scripted bot plays the REAL view play path - the same
    /// Initialize / DriveFrame / TapJob the runtime scene shell runs - second by
    /// second per the requirement-doc beat table:
    /// T+0-3s hook face alive (slogan from the built HUD), T+3.0s the bot taps the
    /// coffee slot and the dispatch lands, the first coin beat answers inside the
    /// 1.0s red-line window through the actual three-sync handler, the work theater
    /// shows the pet on duty, the wage ladder holds one beat per second to T+10s,
    /// and the whole session is exception-free and deterministic across a dual run.
    /// The scene-shell face is asserted by scanning the saved S15_Main (camera +
    /// EventSystem + PetMainHost wired by Scenewire). Refusal / flip comedy faces
    /// stay covered at sim level by the DispatchCheck family - honest scope note.
    /// Exit codes: 0 = all pass, 1 = any fail or exception.
    /// </summary>
    public static class BotBarePlayCheck
    {
        const int Fps = 60;
        const float Dt = 1f / Fps;

        static readonly List<string> Lines = new List<string>();
        static int _passed;
        static int _failed;

        static void Assert(bool ok, string label)
        {
            if (ok) { _passed++; Lines.Add("[PASS] " + label); }
            else { _failed++; Lines.Add("[FAIL] " + label); }
        }

        // one bot session's observable faces (dual-run determinism compares these)
        sealed class Session

```

```

{
    public string Slogan;
    public int IdleAtBoot = -1;
    public int OnDutyAfterTap;
    public int OccupantCoffee = -1;
    public int ChipCount;
    public float FirstFeedbackSec = -1f;
    public int BeatsAt10s;
    public string FlyNumber = "";
    public string CoinText = "";

    public string Key()
    {
        return Slogan + "|" + IdleAtBoot + "|" + OnDutyAfterTap + "|" + OccupantCoffee + "|"
            + ChipCount + "|" + FirstFeedbackSec.ToString("0.000", CultureInfo.InvariantCulture)
            + "|" + BeatsAt10s + "|" + FlyNumber + "|" + CoinText;
    }
}

/// <summary>
/// Scripted bare-play session: hook 0-3s, tap dispatch at T+3.0s, drive the
/// real view path to T+10s. Returns every observable face for the asserts.
/// </summary>
static Session Play()
{
    var go = new GameObject("BotHost");
    var host = go.AddComponent<PetMainHost>();
    host.Initialize(); // real boot path (Awake is silent in edit mode)
    var s = new Session();

    for (int i = 0; i < 3 * Fps; i++) host.DriveFrame(Dt); // T+0-3s hook phase
    s.Slogan = host.SloganFace;
    s.IdleAtBoot = host.BotFirstIdlePet;

    float t = 3f;
    host.TapJob(PetData.JobCoffee); // T+3.0s: the tap a player would make

    while (host.CoinBeatCount == 0 && t < 5f) // red line 11 window with margin
    {
        host.DriveFrame(Dt);
        t += Dt;
    }
    s.FirstFeedbackSec = host.CoinBeatCount > 0 ? t - 3f : -1f;
    s.OnDutyAfterTap = host.OnDutyPets;
    s.OccupantCoffee = host.BotOccupantOf(PetData.JobCoffee);
    s.ChipCount = host.TheaterChipCount;

    while (t < 10f) { host.DriveFrame(Dt); t += Dt; } // ladder to T+10s
    s.BeatsAt10s = host.CoinBeatCount;
    s.FlyNumber = host.LastFlyNumber;
}

```

```

        s.CoinText = host.CoinFace.ToString("0.##", CultureInfo.InvariantCulture);

        Object.DestroyImmediate(go);           // edit-mode teardown, scene stays untouched
        return s;
    }

    /// <summary>Scene shell face: the saved S15_Main is the game, not a stub.</summary>
    static bool ShellScenWired()
    {
        UnityEditor.SceneManagement.EditorSceneManager
            .OpenScene("Assets/_Game/Scenes/S15_Main.unity", UnityEditor.SceneManagement.OpenSceneMode.Single);
        bool host = Object.FindObjectsOfType<PetMainHost>(true).Length > 0;
        bool cam = Object.FindObjectsOfType<Camera>(true).Length > 0;
        bool es = Object.FindObjectsOfType<UnityEngine.EventSystems.EventSystem>(true).Length > 0;
        return host && cam && es;
    }

    public static void Run()
    {
        int code;
        try
        {
            // B1: scene shell face - Scenewire did its job on the saved scene
            Assert(ShellScenWired(), "scene shell wired: camera + EventSystem + PetMainHost in S15_Main");

            var s1 = Play();

            // B2: hook face lives from frame zero (one-line pitch on the HUD)
            Assert(s1.Slogan != null && s1.Slogan == PetMainHost.ComposeHookSlogan() && s1.Slogan.Length > 0,
                "hook face alive from frame zero: '" + s1.Slogan + "'");

            // B3: starter cat idle and dispatchable at boot
            Assert(s1.IdleAtBoot == PetData.PetOrange,
                "starter cat idle at boot (idle pet=" + s1.IdleAtBoot + ")");

            // B4: the tap landed the first dispatch through the real view path
            Assert(s1.OnDutyAfterTap == 1 && s1.OccupantCoffee == PetData.PetOrange,
                "tap dispatch landed: orange cat on duty at coffee");

            // B5: red line 11 hard bound - first feedback <= 1.0s after the tap
            Assert(s1.FirstFeedbackSec >= 0f && s1.FirstFeedbackSec <= PetData.FirstDispatchFeedbackMaxSec,
                "first feedback " + s1.FirstFeedbackSec.ToString("0.000", CultureInfo.InvariantCulture)
                + "s <= " + PetData.FirstDispatchFeedbackMaxSec + "s (delay>1s=fail)");

            // B6: work theater opened - the pet chip reports for duty
            Assert(s1.ChipCount == 1, "work theater open: pet chip on duty (chips=" + s1.ChipCount + ")");

            // B7: three-sync number face fired through the real handler
            Assert(s1.FlyNumber != null && s1.FlyNumber.StartsWith("+"),
                "three-sync number face: '" + s1.FlyNumber + "'");
        }
    }

```

```

// B8: wage ladder to T+10s - one beat per second (3.5..9.5 = 7)
Assert(s1.BeatsAt10s == 7, "wage ladder to T+10s: 7 beats (got " + s1.BeatsAt10s + ")");

// B9: the coin ledger is really rolling
double coin;
bool coinOk = double.TryParse(s1.CoinText, NumberStyles.Float, CultureInfo.InvariantCulture, out coin);
Assert(coinOk && coin > 0.0, "coin ledger rolling at T+10s: " + s1.CoinText);

// B10: bare-play determinism - identical (dt, op) stream, identical faces
var s2 = Play();
Assert(s1.Key() == s2.Key(), "bot session deterministic (dual-run faces identical)");

string projectRoot = Path.GetFullPath(Path.Combine(Application.dataPath, ".."));
var logsDir = Path.Combine(projectRoot, "Logs");
if (!Directory.Exists(logsDir)) Directory.CreateDirectory(logsDir);
var sb = new StringBuilder();
sb.AppendLine("ALL_GREEN passed=" + _passed + " failed=" + _failed);
sb.Append("session=").AppendLine(s1.Key());
foreach (var line in Lines) sb.AppendLine(line);
File.WriteAllText(Path.Combine(logsDir, "botplay_report.txt"), sb.ToString(), new UTF8Encoding(false));
foreach (var line in Lines) Debug.Log(line);
Debug.Log("[BotBarePlayCheck] " + (_failed == 0 ? "ALL_GREEN" : "FAILED") + " passed=" + _passed + " failed=" +
_failed);

    code = _failed == 0 ? 0 : 1;
}
catch (System.Exception e)
{
    Debug.LogError("[BotBarePlayCheck] exception: " + e);
    code = 1;
}
}
EditorApplication.Exit(code);
}
}
}
}
// ===== FILE: Assets\_Game\Scripts\Editor\BotJobBoardCheck.cs =====
using System.Collections.Generic;
using System.Globalization;
using System.IO;
using System.Text;
using Sy.PetWorkCrew.Game;
using UnityEditor;
using UnityEngine;

namespace Sy.PetWorkCrew
{
    /// <summary>
    /// T004 slice B-1b bot gate (batchmode entry, X486 BotRecruitCheck family):
    /// the bot replays the REAL view path - Initialize / DriveFrame / TapJobs /
    /// TapJobCell - through the job board at boot rank R1. Faces asserted: boot

```

```

/// jobs face "Jobs 1/10", the board opens 10 cells with the boot three-face
/// census (coffee open + 9 rank-locked "?? R" faces - every paid post sits
/// behind the shop rank ladder, fire terminal "?? R4"), the rank-locked taps
/// answer with the comedy refusal face and a zero state change, the verdict
/// order mirrors the sim (rank gate first even when coin is also short),
/// the already-open tap folds to a silent no-op, the first-open post (the
/// free coffee) pays a wage ledger isomorphic to a reference pure PetSim fed
/// the identical (dt, op) stream, the census is invariant across the whole
/// refusal family, opening the dex folds the board (one modal at a time),
/// the board toggle closes, and the T002/T003 boot baselines stay intact.
/// Slice C extension (P1-P12): the promote button face reads the five-rank
/// ladder as remaining-gates progress, boot promote taps answer the comedy
/// refusal with zero state change, the star-1 farm flips the face to the
/// ready call-to-action, the promote tap ranks up (R1->R2) with the same-
/// frame celebration fly, the promotion itself opens no post (door, not a
/// grant), the board reopens with paid posts REACHABLE (1 open + 2 priced +
/// 7 locked - the X491-hang payoff), the next-tier rank gate still answers
/// first, the paid courier post opens through the real view path, a second
/// dispatch staffs it, the shared coin-short "Save Nc!" composer holds the
/// cell verdict face (defensive mirror: pack coin:dust coupling makes any
/// promoted state rich, so the cell face stays contract-level), and the
/// full promote chain stays ledger-isomorphic to the reference sim.
/// Exception-free and deterministic across a dual run (byte-identical keys).
/// Report: Logs/botjobboard_report.txt. Exit 0 = all pass.
/// </summary>
public static class BotJobBoardCheck
{
    const int Fps = 60;
    const float Dt = 1f / Fps;
    const float FarmDt = 0.5f; // wage beats are period-locked, granularity does not touch the math

    static readonly List<string> Lines = new List<string>();
    static int _passed;
    static int _failed;

    static void Assert(bool ok, string label)
    {
        if (ok) { _passed++; Lines.Add("[PASS] " + label); }
        else { _failed++; Lines.Add("[FAIL] " + label); }
    }

    // one bot session's observable faces (dual-run determinism compares these)
    sealed class Session
    {
        public string Slogan = "";
        public string RecruitFaceAtBoot = "";
        public string DexFaceAtBoot = "";
        public string JobsFaceAtBoot = "";
        public int BoardCells;
        public string CellCoffee = "";
    }
}

```

```

public string CellCourier = "";
public string CellCarwash = "";
public string CellFish = "";
public string CellFire = "";
public int CensusOpen, CensusPriced, CensusLocked;
public string FishRefusal = "";
public string CoinBeforeFishRefusal = "";
public string CoinAfterFishRefusal = "";
public int JobsAfterFishRefusal;
public string CellFishAfterRefusal = "";
public string CourierRefusal = "";
public string CoinAfterCourierRefusal = "";
public int JobsAfterCourierRefusal;
public string CarwashRefusalWithCoin = "";
public string CoinAfterCarwashRefusal = "";
public int JobsAfterCarwashRefusal;
public string SilentRefusal = "";
public int JobsAfterSilent;
public string CoinAfterBeats = "";
public int BeatsAfterFarm;
public string CoinAfterFarmHost = "";
public string CoinAfterFarmRef = "";
public int CensusOpen2, CensusPriced2, CensusLocked2;
public bool JobsFoldedByDex;
public bool BoardToggledClosed;

// T004 slice C: promote-ladder phase (dual-run determinism compares these too)
public string PromoteFacePre = "";
public string PromoteRefusalPre = "";
public string CoinAfterPromoteRefusal = "";
public int JobsAfterPromoteRefusal;
public string PromoteFaceAfterRecruit = "";
public string PromoteRefusalAfterRecruit = "";
public int StarFarmFrames;
public int StarLevelAfterFarm;
public string PromoteFaceReady = "";
public int RankAfterPromote = -1;
public string PromoteFly = "";
public string PromoteRefusalCleared = "";
public string PromoteFaceAfter = "";
public int JobsAfterPromote;
public int CensusOpen3, CensusPriced3, CensusLocked3;
public string CellCourierPriced = "";
public string FishRefusalR2 = "";
public int JobsAfterCourierOpen;
public string CellCourierOpened = "";
public int OnDutyAfterDispatch2;
public string SaveContract = "";
public string CoinIsoHost2 = "";
public string CoinIsoRef2 = "";

```

```

public int RankIsoRef = -1;

public string Key()
{
    return Slogan + "|" + RecruitFaceAtBoot + "|" + DexFaceAtBoot + "|" + JobsFaceAtBoot + "|"
        + BoardCells + "|" + CellCoffee + "|" + CellCourier + "|" + CellCarwash + "|"
        + CellFish + "|" + CellFire + "|"
        + CensusOpen + "|" + CensusPriced + "|" + CensusLocked + "|"
        + FishRefusal + "|" + CoinBeforeFishRefusal + "|" + CoinAfterFishRefusal + "|"
        + JobsAfterFishRefusal + "|" + CellFishAfterRefusal + "|"
        + CourierRefusal + "|" + CoinAfterCourierRefusal + "|" + JobsAfterCourierRefusal + "|"
        + CarwashRefusalWithCoin + "|" + CoinAfterCarwashRefusal + "|" + JobsAfterCarwashRefusal + "|"
        + SilentRefusal + "|" + JobsAfterSilent + "|"
        + CoinAfterBeats + "|" + BeatsAfterFarm + "|" + CoinAfterFarmHost + "|" + CoinAfterFarmRef + "|"
        + CensusOpen2 + "|" + CensusPriced2 + "|" + CensusLocked2 + "|"
        + JobsFoldedByDex + "|" + BoardToggledClosed + "|"
        + PromoteFacePre + "|" + PromoteRefusalPre + "|" + CoinAfterPromoteRefusal + "|"
        + JobsAfterPromoteRefusal + "|" + PromoteFaceAfterRecruit + "|"
        + PromoteRefusalAfterRecruit + "|" + StarFarmFrames + "|" + StarLevelAfterFarm + "|"
        + PromoteFaceReady + "|" + RankAfterPromote + "|" + PromoteFly + "|"
        + PromoteRefusalCleared + "|" + PromoteFaceAfter + "|" + JobsAfterPromote + "|"
        + CensusOpen3 + "|" + CensusPriced3 + "|" + CensusLocked3 + "|"
        + CellCourierPriced + "|" + FishRefusalR2 + "|" + JobsAfterCourierOpen + "|"
        + CellCourierOpened + "|" + OnDutyAfterDispatch2 + "|" + SaveContract + "|"
        + CoinIsoHost2 + "|" + CoinIsoRef2 + "|" + RankIsoRef;
}

}

static void Census(PetMainHost host, out int open, out int priced, out int locked)
{
    open = 0; priced = 0; locked = 0;
    for (int j = 0; j < PetData.JobCount; j++)
    {
        string face = host.JobCellFace(j);
        if (face.EndsWith(" open")) open++;
        else if (face.StartsWith("Open ")) priced++;
        else if (face.StartsWith("??? R")) locked++;
    }
}

/// <summary>
/// Scripted board-play session through the real view path. The host and a
/// reference pure PetSim receive the identical (dt, op) stream, so every
/// coin readout must match the offline ledger to the last bit.
/// </summary>
static Session Play()
{
    var go = new GameObject("BotBoardHost");
    var host = go.AddComponent<PetMainHost>();
    host.Initialize(); // real boot path (Awake is silent in edit mode)
}

```

```

var refSim = new PetSim();
refSim.SeedStarterState();
var s = new Session();

void Frame(float dt)
{
    host.DriveFrame(dt);
    refSim.Tick(dt);
}

string R(double v) => v.ToString("R", CultureInfo.InvariantCulture);

s.Slogan = host.SloganFace;
s.RecrUITFaceAtBoot = host.RecrUITButtonFace;
s.DexFaceAtBoot = host.DexButtonFace;
s.JobsFaceAtBoot = host.JobsButtonFace;

// board opens: 10 cells with the boot three-face census
host.TapJobs();
s.BoardCells = host.JobCellFaceCount;
s.CellCoffee = host.JobCellFace(PetData.JobCoffee);
s.CellCourier = host.JobCellFace(PetData.JobCourier);
s.CellCarwash = host.JobCellFace(PetData.JobCarwash);
s.CellFish = host.JobCellFace(PetData.JobFish);
s.CellFire = host.JobCellFace(PetData.JobFire);
Census(host, out s.CensusOpen, out s.CensusPriced, out s.CensusLocked);

// rank-locked comedy refusals: zero state change (coin + jobs + cell face)
string coinBefore = R(host.CoinFace);
host.TapJobCell(PetData.JobFish); // R2 post at boot rank R1
s.FishRefusal = host.LastJobRefusal;
s.CoinBeforeFishRefusal = coinBefore;
s.CoinAfterFishRefusal = R(host.CoinFace);
s.JobsAfterFishRefusal = host.BotJobsOpenCount;
s.CellFishAfterRefusal = host.JobCellFace(PetData.JobFish);
host.TapJobCell(PetData.JobCourier); // cheapest tier R1 post
s.CourierRefusal = host.LastJobRefusal;
s.CoinAfterCourierRefusal = R(host.CoinFace);
s.JobsAfterCourierRefusal = host.BotJobsOpenCount;

// dispatch the first-open post (free coffee) then farm real wage beats
for (int i = 0; i < 3 * Fps; i++) Frame(Dt); // T+0-3s hook phase (zero wage)
host.TapJob(PetData.JobCoffee); // T+3.0s dispatch tap (baseline continuity)
refSim.AssignPet(PetData.JobCoffee, PetData.PetOrange);
for (int i = 0; i < 4; i++) Frame(FarmDt); // first wage beats land, coin << 280
s.BeatsAfterFarm = host.CoinBeatCount;
s.CoinAfterBeats = R(host.CoinFace);

// coin is ALSO short now, but the rank gate still answers first
// (verdict order mirrors the sim: rank first, coin second)

```

```

host.TapJobCell(PetData.JobCarwash);
s.CarwashRefusalWithCoin = host.LastJobRefusal;
s.CoinAfterCarwashRefusal = R(host.CoinFace);
s.JobsAfterCarwashRefusal = host.BotJobsOpenCount;

// already-open post folds to a silent no-op
host.TapJobCell(PetData.JobCoffee);
s.SilentRefusal = host.LastJobRefusal;
s.JobsAfterSilent = host.BotJobsOpenCount;

// long farm through the first-open post wage ladder: ledger isomorphism
int guard = 0;
while (host.CoinFace < 280.0 && guard++ < 4000) Frame(FarmDt);
s.CoinAfterFarmHost = R(host.CoinFace);
s.CoinAfterFarmRef = R(refSim.Coin);
Census(host, out s.CensusOpen2, out s.CensusPriced2, out s.CensusLocked2);

// one modal at a time: opening the dex folds the board; board toggle closes
host.TapDex();
s.JobsFoldedByDex = !host.JobsOpen && host.DexOpen;
host.TapDex();
host.TapJobs();
host.TapJobs();
s.BoardToggledClosed = !host.JobsOpen;

// ----- T004 slice C: promote ladder through the real view path -----
s.PromoteFacePre = host.PromoteButtonFace;
host.TapPromote(); // gates not met -> comedy refusal
s.PromoteRefusalPre = host.LastPromoteRefusal;
s.CoinAfterPromoteRefusal = R(host.CoinFace);
s.JobsAfterPromoteRefusal = host.BotJobsOpenCount;

host.TapRecruit(); // snowball 200c closes the pal gate
refSim.RecruitPet(1);
s.PromoteFaceAfterRecruit = host.PromoteButtonFace;
host.TapPromote();
s.PromoteRefusalAfterRecruit = host.LastPromoteRefusal;

int starGuard = 0;
while (host.BotStarLevel < 1 && starGuard++ < 40000) Frame(FarmDt); // star-1 dust farm
s.StarFarmFrames = starGuard;
s.StarLevelAfterFarm = host.BotStarLevel;
s.PromoteFaceReady = host.PromoteButtonFace;

host.TapPromote(); // gates met -> rank up R1->R2
refSim.TryPromote();
s.RankAfterPromote = host.BotRankIndex;
s.PromoteFly = host.LastPromoteFly;
s.PromoteRefusalCleared = host.LastPromoteRefusal;
s.PromoteFaceAfter = host.PromoteButtonFace;

```

```

s.JobsAfterPromote = host.BotJobsOpenCount;

host.TapJobs(); // board reopens: paid posts reachable
Census(host, out s.CensusOpen3, out s.CensusPriced3, out s.CensusLocked3);
s.CellCourierPriced = host.JobCellFace(PetData.JobCourier);

host.TapJobCell(PetData.JobFish); // next-tier gate still answers first
s.FishRefusalR2 = host.LastJobRefusal;

host.TapJobCell(PetData.JobCourier); // paid post opens through the view
refSim.OpenJob(PetData.JobCourier);
s.JobsAfterCourierOpen = host.BotJobsOpenCount;
s.CellCourierOpened = host.JobCellFace(PetData.JobCourier);

host.TapJob(PetData.JobCourier); // second dispatch staffs it
refSim.AssignPet(PetData.JobCourier, 1);
s.OnDutyAfterDispatch2 = host.OnDutyPets;

s.SaveContract = PetMainHost.ComposeSaveRefusal(280.0); // cell coin face contract
s.CoinIsoHost2 = R(host.CoinFace);
s.CoinIsoRef2 = R(refSim.Coin);
s.RankIsoRef = refSim.RankIndex;

Object.DestroyImmediate(go); // edit-mode teardown, scene stays untouched
return s;
}

[MenuItem("Sy/G15/BotJobBoardCheck")]
public static void Run()
{
    int code;
    try
    {
        var s1 = Play();

        // J1: boot jobs face counts the free coffee open
        Assert(s1.JobsFaceAtBoot == "Jobs 1/10",
            "boot jobs face 'Jobs 1/10': '" + s1.JobsFaceAtBoot + "'");

        // J2: the board opens with all 10 job cells
        Assert(s1.BoardCells == PetData.JobCount,
            "board opens 10 cells (count=" + s1.BoardCells + ")");

        // J3: boot cell faces - first-open post live, paid posts rank-locked
        Assert(s1.CellCoffee == "coffee open" && s1.CellCourier == "???" R1"
            && s1.CellCarwash == "???" R1" && s1.CellFish == "???" R2",
            "boot cell faces: '" + s1.CellCoffee + "'/'" + s1.CellCourier + "'/'"
            + s1.CellCarwash + "'/'" + s1.CellFish + "'");

        // J4: fire station terminal lock face
    }
}

```

(middle pages omitted)

pages 31 - 207 of 237 omitted per first-30 + last-30 excerpt rule.

```

    {
        float t = i / (float)rate;
        int step = Mathf.Min(2, (int)(t / 0.22f)); // horn burst ladder
        float seg = t - step * 0.22f;
        float env = seg < 0.22f ? Mathf.Max(0f, 1f - seg * 5.5f) : 0f; // burst decay
        float tone = Mathf.Sin(2f * Mathf.PI * horn[step] * t)
            + 0.35f * Mathf.Sin(4f * Mathf.PI * horn[step] * t); // brassy edge
        if (t > 0.66f) // celebrating bell tail
            tone += 0.6f * Mathf.Sin(2f * Mathf.PI * 1568f * t)
                * Mathf.Max(0f, 1f - (t - 0.66f) * 3f);
        buf[i] = env * tone * 0.5f;
    }
    clip.SetData(buf, 0);
    return clip;
}
catch (System.Exception)
{
    return null; // batchmode/-nographics safe
}
}

void BuildUi()
{
    var canvas = GetComponent<Canvas>() != null
        ? GetComponent<Canvas>()
        : gameObject.AddComponent<Canvas>();
    if (canvas.renderMode == RenderMode.WorldSpace) canvas.renderMode = RenderMode.ScreenSpaceOverlay;
    if (GetComponent<CanvasScaler>() == null) gameObject.AddComponent<CanvasScaler>();
    if (GetComponent<GraphicRaycaster>() == null) gameObject.AddComponent<GraphicRaycaster>();

    // T+0-3s hook face: slogan lives from frame zero (coin rain = placeholder beats)
    _sloganText = MakeText(canvas.transform, ComposeHookSlogan(), 34, new Vector2(0f, 260f), Color.white);

    // coin readout (number face of the three-sync)
    var readout = MakePanel(canvas.transform, new Vector2(0f, 190f), new Vector2(420f, 64f), new Color(0.09f, 0.07f, 0.1f,
0.85f));
    _coinReadout = readout.transform;
    _coinText = MakeText(readout.transform, "0", 40, Vector2.zero, new Color(1f, 0.82f, 0.25f));

    // flying wage number (the "+x.xx" that pops on every beat)
    _flyText = MakeText(canvas.transform, "", 30, new Vector2(0f, 130f), new Color(1f, 0.95f, 0.5f));

    // T003 slice B: recruit button (coin direct-buy, zero gacha) + dex button
    var recruitGo = MakePanel(canvas.transform, new Vector2(330f, 190f),
        new Vector2(220f, 64f), new Color(0.85f, 0.55f, 0.15f, 0.95f));
    recruitGo.AddComponent<Button>().onClick.AddListener(TapRecruit);
    _recruitLabel = MakeText(recruitGo.transform, "", 22, Vector2.zero, Color.black);
    var dexGo = MakePanel(canvas.transform, new Vector2(330f, 116f),
        new Vector2(220f, 64f), new Color(0.2f, 0.45f, 0.75f, 0.95f));
    dexGo.AddComponent<Button>().onClick.AddListener(TapDex);
    _dexCountFace = MakeText(dexGo.transform, "", 22, Vector2.zero, Color.white);
}

```

```

// T004 slice B: job board button (10 posts, open chain)
var jobsGo = MakePanel(canvas.transform, new Vector2(330f, 42f),
    new Vector2(220f, 64f), new Color(0.55f, 0.3f, 0.6f, 0.95f));
jobsGo.AddComponent<Button>().onClick.AddListener(TapJobs);
_jobsLabel = MakeText(jobsGo.transform, "", 22, Vector2.zero, Color.white);

// T004 slice C: shop promote button (five-rank ladder door, gold face)
var promoteGo = MakePanel(canvas.transform, new Vector2(330f, -32f),
    new Vector2(220f, 64f), new Color(0.9f, 0.66f, 0.12f, 0.95f));
promoteGo.AddComponent<Button>().onClick.AddListener(TapPromote);
_promoteLabel = MakeText(promoteGo.transform, "", 19, Vector2.zero, Color.black);

// T007 slice B: rescue offer seat (peak short-staff rescue ad, teal face)
var rescueGo = MakePanel(canvas.transform, new Vector2(330f, -106f),
    new Vector2(220f, 64f), new Color(0.15f, 0.5f, 0.5f, 0.95f));
rescueGo.AddComponent<Button>().onClick.AddListener(TapRescueOffer);
_rescueLabel = MakeText(rescueGo.transform, "", 18, Vector2.zero, Color.white);

// T008 slice B-1: postcard album button (journal count face)
var cardsGo = MakePanel(canvas.transform, new Vector2(330f, -180f),
    new Vector2(220f, 64f), new Color(0.72f, 0.5f, 0.28f, 0.95f));
cardsGo.AddComponent<Button>().onClick.AddListener(TapCards);
_cardsCountFace = MakeText(cardsGo.transform, "", 22, Vector2.zero, Color.black);

// T012: honors button (honor wall, achievement count face)
var honorsGo = MakePanel(canvas.transform, new Vector2(330f, -254f),
    new Vector2(220f, 64f), new Color(0.52f, 0.4f, 0.72f, 0.95f));
honorsGo.AddComponent<Button>().onClick.AddListener(TapHonors);
_honorsLabel = MakeText(honorsGo.transform, "", 19, Vector2.zero, Color.white);

// T007 slice B: offline x2 re-presentation chip (top-left seat; armed by
// TapOfflineClose while the slot is pending, tap reopens the receipt page)
_doubleChipGo = MakePanel(canvas.transform, new Vector2(-330f, 190f),
    new Vector2(200f, 56f), new Color(0.9f, 0.55f, 0.1f, 0.95f));
_doubleChipGo.AddComponent<Button>().onClick.AddListener(TapDoubleChip);
_doubleChipFace = MakeText(_doubleChipGo.transform, "", 20, Vector2.zero, Color.black);

// T011: HUD shop stage name face (M04 five-stage re-skin, renames on promote)
_shopStageFace = MakeText(canvas.transform, "", 18, new Vector2(-330f, 132f),
    new Color(1f, 0.82f, 0.25f));

// work-theater placeholder row: one slot per unlocked job (U020 D-grade faces)
var row = MakePanel(canvas.transform, new Vector2(0f, -40f), new Vector2(900f, 320f), new Color(0.12f, 0.1f, 0.14f,
0.8f));
_theaterRow = row.transform;
RebuildTheaterRow();

// T003 slice B: pet dex page overlay (modal - built last so it draws on top)
_dexPage = MakePanel(canvas.transform, Vector2.zero, new Vector2(760f, 430f),
    new Color(0.07f, 0.09f, 0.13f, 0.97f));

```

```

MakeText(_dexPage.transform, "Pet Dex", 30, new Vector2(0f, 180f), new Color(1f, 0.82f, 0.25f));
_dexPage.SetActive(false);

// T004 slice B: job board page overlay (built after dex; one modal at a time)
_jobsPage = MakePanel(canvas.transform, Vector2.zero, new Vector2(760f, 430f),
    new Color(0.1f, 0.08f, 0.14f, 0.97f));
MakeText(_jobsPage.transform, "Job Posts", 30, new Vector2(0f, 180f), new Color(1f, 0.82f, 0.25f));
_jobsPage.SetActive(false);

// T008 slice B-1: postcard album page (one modal at a time; channel B
// share-code + import faces live on this page)
_postcardPage = MakePanel(canvas.transform, Vector2.zero, new Vector2(760f, 430f),
    new Color(0.13f, 0.1f, 0.08f, 0.97f));
MakeText(_postcardPage.transform, "Postcards", 30, new Vector2(0f, 180f), new Color(1f, 0.82f, 0.25f));
_postcardCellHint = MakeText(_postcardPage.transform, "", 18, new Vector2(0f, 148f),
    new Color(0.75f, 0.75f, 0.78f));
var shareGo = MakePanel(_postcardPage.transform, new Vector2(-150f, -130f), new Vector2(180f, 52f),
    new Color(0.2f, 0.45f, 0.75f, 0.95f));
shareGo.AddComponent<Button>().onClick.AddListener(TapShareCode);
MakeText(shareGo.transform, "Share code", 17, Vector2.zero, Color.white);
var importGo = MakePanel(_postcardPage.transform, new Vector2(150f, -130f), new Vector2(180f, 52f),
    new Color(0.35f, 0.4f, 0.35f, 0.95f));
importGo.AddComponent<Button>().onClick.AddListener(TapImportCode);
MakeText(importGo.transform, "Import", 17, Vector2.zero, Color.white);
var saveGo = MakePanel(_postcardPage.transform, new Vector2(0f, -130f), new Vector2(110f, 52f),
    new Color(0.85f, 0.62f, 0.18f, 0.95f)); // T008 B-2: channel A share-image seat
saveGo.AddComponent<Button>().onClick.AddListener(TapShareImage);
MakeText(saveGo.transform, "Save", 17, Vector2.zero, Color.black);
_postcardCodeFace = MakeText(_postcardPage.transform, "", 14, new Vector2(0f, -170f),
    new Color(0.85f, 0.95f, 1f));
_postcardImportFace = MakeText(_postcardPage.transform, "", 16, new Vector2(0f, -196f),
    new Color(1f, 0.9f, 0.6f));
_postcardPage.SetActive(false);

// T012: honor wall page (SYS08; 8 achievement cells, one modal at a time)
_honorPage = MakePanel(canvas.transform, Vector2.zero, new Vector2(760f, 430f),
    new Color(0.09f, 0.07f, 0.13f, 0.97f));
MakeText(_honorPage.transform, ComposeHonorWallTitle(), 30, new Vector2(0f, 180f),
    new Color(1f, 0.82f, 0.25f));
_honorPage.SetActive(false);

// T005 slice B: offline return-visit page (built last so the receipt draws on top)
_offlinePage = MakePanel(canvas.transform, Vector2.zero, new Vector2(760f, 430f),
    new Color(0.08f, 0.12f, 0.09f, 0.97f));
_offlineWelcomeFace = MakeText(_offlinePage.transform, "", 26, new Vector2(0f, 120f),
    new Color(1f, 0.82f, 0.25f));
_offlineMailFace = MakeText(_offlinePage.transform, "", 20, new Vector2(0f, 78f),
    new Color(1f, 0.9f, 0.6f)); // T008: the away summary postcard strip
_offlineGainFace = MakeText(_offlinePage.transform, "", 34, new Vector2(0f, 30f), Color.white);
var doubleGo = MakePanel(_offlinePage.transform, new Vector2(0f, -80f), new Vector2(320f, 72f),

```

```

        new Color(0.9f, 0.55f, 0.1f, 0.95f));
doubleGo.AddComponent<Button>().onClick.AddListener(TapOfflineDouble);
_offlineDoubleFace = MakeText(doubleGo.transform, "", 26, Vector2.zero, Color.black);
var collectGo = MakePanel(_offlinePage.transform, new Vector2(0f, -170f), new Vector2(220f, 56f),
    new Color(0.35f, 0.4f, 0.35f, 0.95f));
collectGo.AddComponent<Button>().onClick.AddListener(TapOfflineClose);
MakeText(collectGo.transform, "Collect", 22, Vector2.zero, Color.white);
_offlinePage.SetActive(false);

// T011: promotion stamp ceremony overlay (SYS05; draws above the pages
// and below the event card - the interrupt card law keeps top draw, so
// a comedy pop during the ceremony stays collectable above it). The
// ceremony never closes another page: it is a floating celebration.
_ceremonyPage = MakePanel(canvas.transform, Vector2.zero, new Vector2(760f, 430f),
    new Color(0.12f, 0.07f, 0.03f, 0.97f));
_ceremonyTitleFace = MakeText(_ceremonyPage.transform, "", 32, new Vector2(0f, 150f),
    new Color(1f, 0.82f, 0.25f));
_ceremonyStageFace = MakeText(_ceremonyPage.transform, "", 22, new Vector2(0f, 100f),
    new Color(1f, 0.9f, 0.6f));
_ceremonyStampFace = MakeText(_ceremonyPage.transform, "", 40, new Vector2(0f, 40f),
    new Color(1f, 0.82f, 0.25f));
_ceremonyUnlockFace = MakeText(_ceremonyPage.transform, "", 22, new Vector2(0f, -30f),
    Color.white);
_ceremonyCrewFace = MakeText(_ceremonyPage.transform, "", 18, new Vector2(0f, -75f),
    new Color(0.75f, 0.75f, 0.78f));
var ceremonyCollectGo = MakePanel(_ceremonyPage.transform,
    new Vector2(0f, -160f), new Vector2(240f, 56f), new Color(0.35f, 0.4f, 0.35f, 0.95f));
ceremonyCollectGo.AddComponent<Button>().onClick.AddListener(TapCeremonyCollect);
_ceremonyCollectLabel = MakeText(ceremonyCollectGo.transform, "", 20, Vector2.zero,
    Color.white);
_ceremonyPage.SetActive(false);

// T013: endgame finale overlay (M10; floating celebration like the
// ceremony - draws above the pages, never closes another page). The
// all-lit stamp + crew parade beats are frame-driven; the collect tap
// is pure applause (the ending already landed in the sim).
_endgamePage = MakePanel(canvas.transform, new Vector2(0f, 40f), new Vector2(760f, 430f),
    new Color(0.05f, 0.08f, 0.12f, 0.97f));
_endgameTitleFace = MakeText(_endgamePage.transform, "", 32, new Vector2(0f, 150f),
    new Color(1f, 0.82f, 0.25f));
_endgameAllLitFace = MakeText(_endgamePage.transform, "", 40, new Vector2(0f, 40f),
    new Color(1f, 0.82f, 0.25f));
_endgameCrewFace = MakeText(_endgamePage.transform, "", 18, new Vector2(0f, -60f),
    new Color(0.75f, 0.75f, 0.78f));
var endgameCollectGo = MakePanel(_endgamePage.transform,
    new Vector2(-140f, -160f), new Vector2(240f, 56f), new Color(0.35f, 0.4f, 0.35f, 0.95f));
endgameCollectGo.AddComponent<Button>().onClick.AddListener(TapEndgameCollect);
_endgameCollectLabel = MakeText(endgameCollectGo.transform, "", 20, Vector2.zero,
    Color.white);
// T013 slice B: the annual report share seat rides the finale overlay

```

```

// (side by side with the collect button - the ending IS the share moment)
var endgameShareGo = MakePanel(_endgamePage.transform,
    new Vector2(140f, -160f), new Vector2(240f, 56f), new Color(0.36f, 0.3f, 0.16f, 0.95f));
endgameShareGo.AddComponent<Button>().onClick.AddListener(TapShareAnnual);
_endgameShareLabel = MakeText(endgameShareGo.transform, "", 20, Vector2.zero,
    Color.white);
// T013 slice D: the new-run seat rides the finale overlay - the warm
// gold forward path between the crew parade and the bottom row (the
// collections carry, the ladder resets: the "what's next" hook)
var endgameNgPlusGo = MakePanel(_endgamePage.transform,
    new Vector2(0f, -105f), new Vector2(300f, 44f), new Color(0.5f, 0.38f, 0.12f, 0.95f));
endgameNgPlusGo.AddComponent<Button>().onClick.AddListener(TapNgPlusRun);
_endgameNgPlusLabel = MakeText(endgameNgPlusGo.transform, "", 20, Vector2.zero,
    Color.white);
_endgamePage.SetActive(false);

// T006 slice B: comedy event card pop (built LAST so the beat draws above
// every page; the card never closes another modal - the offline x2 offer
// must stay reachable - and the whole card is the tap-to-collect face)
_eventCardPage = MakePanel(canvas.transform, new Vector2(0f, 70f), new Vector2(600f, 260f),
    new Color(0.16f, 0.11f, 0.06f, 0.98f));
_eventCardPage.AddComponent<Button>().onClick.AddListener(TapEventCard);
_eventCardTitleFace = MakeText(_eventCardPage.transform, "", 26, new Vector2(0f, 55f),
    new Color(1f, 0.82f, 0.25f));
_eventCardGainFace = MakeText(_eventCardPage.transform, "", 34, new Vector2(0f, -25f), Color.white);
MakeText(_eventCardPage.transform, "Tap to collect", 18, new Vector2(0f, -92f),
    new Color(0.75f, 0.75f, 0.78f));
_eventCardPage.SetActive(false);
}

void RebuildTheaterRow()
{
    for (int i = _petChips.Count - 1; i >= 0; i--) DestroyFace(_petChips[i].gameObject);
    _petChips.Clear();
    for (int i = _speedCellTexts.Count - 1; i >= 0; i--)
        if (_speedCellTexts[i] != null) DestroyFace(_speedCellTexts[i].gameObject);
    _speedCellTexts.Clear();
    for (int j = 0; j < PetData.JobCount; j++) _speedCellTexts.Add(null);
    bool anyCallableSeat = false;
    string day = Sy.Platform.AdCapPolicy.TodayKey();
    bool speedBudget = _adCap != null
        && _adCap.HasCategoryBudget(day, Sy.Platform.AdCategories.G15Speed);
    int slotIndex = 0;
    for (int j = 0; j < PetData.JobCount; j++)
    {
        if (!_sim.IsJobUnlocked(j)) continue;
        var open = ComposeTheaterOpen(j, _sim.JobOccupant(j));
        var slot = MakePanel(_theaterRow, new Vector2(-260f + 200f * slotIndex, 0f), new Vector2(180f, 280f), new
Color(0.16f, 0.14f, 0.2f, 0.95f));
        var btn = slot.AddComponent<Button>(); // slice B-1c: the slot itself is the tap face
        int tapped = j;

```

```

        btn.onClick.AddListener(() => TapJob(tapped));
        slotIndex++;
        MakeText(slot.transform, open.JobLabel, 26, new Vector2(0f, 100f), Color.white);
        if (_sim.JobOccupant(j) >= 0)
        {
            var chip = MakePanel(slot.transform, new Vector2(0f, -10f), new Vector2(96f, 96f),
PetChipColor(_sim.JobOccupant(j)));
            MakeText(chip.transform, open.PetLabel, 22, Vector2.zero, Color.black);
            _petChips.Add(chip.transform);
            // T007 slice B: speed offer seat on the occupied slot only - an
            // empty job can never waste an ad view (pre-offer gate by design).
            bool boosted = _sim.SpeedBoostActive(j);
            if (!boosted && speedBudget && _sim.SpeedBoostJob < 0) anyCallableSeat = true;
            var speedGo = MakePanel(slot.transform, new Vector2(0f, -108f), new Vector2(120f, 44f),
                boosted ? new Color(0.2f, 0.62f, 0.28f, 0.95f) : new Color(0.85f, 0.45f, 0.12f, 0.95f));
            speedGo.AddComponent<Button>().onClick.AddListener(() => TapSpeedOffer(tapped));
            _speedCellTexts[j] = MakeText(speedGo.transform,
                ComposeSpeedCellFace(boosted), 15, Vector2.zero, Color.white);
        }
    }
    // evt_ad offer state for the speed family: one write per visible
    // callable epoch (any occupied seat with budget and no live window)
    if (anyCallableSeat && !_speedOfferShown)
    {
        _speedOfferShown = true;
        Funnel("offer", SpeedPointId);
    }
    else if (!anyCallableSeat) _speedOfferShown = false;
}

static void DestroyFace(Object go)
{
    // edit-mode safe teardown (bot gate rebuilds the row in batchmode)
    if (Application.isPlaying) Destroy(go);
    else DestroyImmediate(go);
}

static Color PetChipColor(int petId)
{
    // placeholder pet chip tint by pet index (catalog sprite swap point)
    float h = (petId * 0.13f) % 1f;
    return Color.HSVToRGB(h, 0.55f, 0.95f);
}

void HandleCoinBeat(float t, double delta, int jobId)
{
    // three-sync: sound + pulse + flying number, all in this same frame
    var fb = ComposeCoinFeedback(t, delta, jobId);
    _coinBeatCount++;
    _lastFlyNumber = fb.NumberText;
    // audible face only at play time; the edit-mode bot gate skips it

```

```

    // (the ding contract itself is asserted by Redline11Check)
    if (_dingClip != null && Application.isPlaying) _ding.PlayOneShot(_dingClip);
    _pulseAge = 0f;
    _flyAge = 0f;
    _flyText.text = fb.NumberText;
}

void HandleDispatched(int jobId, int petId, float t)
{
    // work theater opens on the dispatch frame (pet reports for duty)
    RebuildTheaterRow();
}

void Update()
{
    DriveFrame(Time.deltaTime);
}

/// <summary>One real view frame; the bot gate drives the same body with fixed dt.</summary>
public void DriveFrame(float dt)
{
    _sim.Tick(dt);

    // T007 slice B: boost epoch flip (window open <-> closed) rebuilds the
    // theater speed seats + rescue face; per-second rescue countdown refresh
    bool epoch = _sim.RescueBoostActive || _sim.SpeedBoostJob >= 0;
    if (epoch != _boostEpochSeen)
    {
        _boostEpochSeen = epoch;
        RebuildTheaterRow();
        RefreshBoostFaces();
    }
    int faceSec = (int)_sim.SimTime;
    if (faceSec != _lastFaceSec)
    {
        _lastFaceSec = faceSec;
        RefreshBoostFaces();
    }
    // T011 ceremony show beats: the stamp slams, then the unlock + crew
    // faces pop - deterministic frame-driven clocks, one flip each
    if (_ceremonyOpen)
    {
        _ceremonyT += dt;
        int newRank = _sim != null ? _sim.RankIndex : 0;
        if (!_ceremonyStampShown && _ceremonyT >= CeremonyStampAtSec)
        {
            _ceremonyStampShown = true;
            _ceremonyStampFace.text = ComposeCeremonyStampFace(newRank);
        }
        if (!_ceremonyUnlockShown && _ceremonyT >= CeremonyUnlockAtSec)

```

```

    {
        _ceremonyUnlockShown = true;
        _ceremonyUnlockFace.text = ComposeCeremonyUnlockFace(newRank);
        _ceremonyCrewFace.text = ComposeCeremonyCrewFace(newRank);
    }
}

// T013 finale show beats: the all-lit stamp slams, then the crew parade
// face pops - deterministic frame-driven clocks, one flip each
if (_endgameOpen)
{
    _endgameT += dt;
    if (!_endgameAllLitShown && _endgameT >= EndgameAllLitAtSec)
    {
        _endgameAllLitShown = true;
        _endgameAllLitFace.text = ComposeEndgameAllLitFace(_sim.JobsOpenCount);
    }
    if (!_endgameCrewShown && _endgameT >= EndgameCrewAtSec)
    {
        _endgameCrewShown = true;
        var crew = new System.Collections.Generic.List<string>();
        for (int p = 0; p < PetData.PetCount; p++)
            if (_sim.IsPetRecruited(p)) crew.Add(PetData.PetIds[p].Substring(4));
        _endgameCrewFace.text = ComposeEndgameCrewFace(string.Join(", ", crew.ToArray()));
    }
}

// x2 chip presentation window (pack section 7 "1-5/200" law): blank in
// the first second after the close, loud inside the 200s window, quiet
// badge after - tappable in every state until the slot is claimed
if (_sim.HasDoubleOfferPending && _chipArmedAt >= 0.0 && !_offlineOpen)
{
    string chipFace = ComposeDoubleChipFace(_sim.SimTime - _chipArmedAt);
    if (_doubleChipFace.text != chipFace) _doubleChipFace.text = chipFace;
}
else if (_doubleChipFace.text.Length > 0) _doubleChipFace.text = "";

// number face: the readout rolls toward the ledger value
if (_coinShown < _sim.Coin)
{
    _coinShown += (_sim.Coin - _coinShown) * Mathf.Min(1f, 18f * dt);
    if (_sim.Coin - _coinShown < 0.005) _coinShown = _sim.Coin;
    _coinText.text = _coinShown.ToString("0.##", CultureInfo.InvariantCulture);
}

// pulse face decay
if (_pulseAge < PulseDur)
{
    _pulseAge += dt;
    float k = 1f - (_pulseAge / PulseDur);

```

```

        if (k < 0f) k = 0f;
        _coinReadout.localScale = Vector3.one * (1f + 0.35f * k);
    }
    else _coinReadout.localScale = Vector3.one;

    // flying number face
    if (_flyAge < FlyDur)
    {
        _flyAge += dt;
        float k = _flyAge / FlyDur;
        _flyText.rectTransform.anchoredPosition = new Vector2(0f, 130f + 90f * k);
        var c = _flyText.color;
        c.a = 1f - k;
        _flyText.color = c;
    }
    else if (_flyText.text.Length > 0) _flyText.text = "";

    // theater placeholder: occupied pet chips hop (working comedy face)
    for (int i = 0; i < _petChips.Count; i++)
    {
        if (_petChips[i] == null) continue;
        float hop = Mathf.Abs(Mathf.Sin((float)_sim.SimTime * 3.5f + i)) * 14f;
        _petChips[i].localPosition = new Vector3(0f, -10f + hop, 0f);
    }
}

// ----- procedural D-grade placeholder builders (U020; catalog swap later) -----
static GameObject MakePanel(Transform parent, Vector2 pos, Vector2 size, Color color)
{
    var go = new GameObject("panel", typeof(RectTransform), typeof(Image));
    go.transform.SetParent(parent, false);
    var rt = go.GetComponent<RectTransform>();
    rt.anchoredPosition = pos;
    rt.sizeDelta = size;
    var img = go.GetComponent<Image>();
    img.color = color;
    return go;
}

static Text MakeText(Transform parent, string content, int size, Vector2 pos, Color color)
{
    var go = new GameObject("label", typeof(RectTransform), typeof(Text));
    go.transform.SetParent(parent, false);
    var rt = go.GetComponent<RectTransform>();
    rt.anchoredPosition = pos;
    rt.sizeDelta = new Vector2(800f, size + 12f);
    var txt = go.GetComponent<Text>();
    txt.text = content;
    txt.fontSize = size;
    txt.color = color;
}

```

```

        txt.alignment = TextAnchor.MiddleCenter;
        txt.font = Resources.GetBuiltinResource<Font>("LegacyRuntime.ttf");
        return txt;
    }
}
// ===== FILE: Assets\_Game\Scripts\Runtime\Game\PetSim.cs =====
using System;
using System.Globalization;
using System.Collections.Generic;
using System.Text;

namespace Sy.PetWorkCrew.Game
{
    /// <summary>
    /// T002 slice A: dispatch core loop simulation (pure C#, zero scene dependency,
    /// zero RNG - idle determinism). Law: state is a pure function of the (dt stream,
    /// op stream), so any two runs with identical inputs produce byte-identical
    /// fingerprints - the base the offline settle (T005) later proves isomorphic
    /// against. Red line 11 anchors owned by this slice: hook content lives from
    /// frame zero, the first dispatch answers with its first wage beat inside the
    /// 1.0s window with margin (pack section 9), every occupied job produces per
    /// the rate table with the log soft cap, empty jobs produce nothing and nothing
    /// ever punishes (flip-a-pet is a comedy beat, no dead end). Slice B (next
    /// round) wires the View (coin ding three-sync: sound + pulse + number, work
    /// theater placeholder) and the DispatchCheck / Redline11Check editor gates.
    /// </summary>
    public sealed class PetSim
    {
        /// <summary>Wage settle cadence: one coin beat per second per occupied job.</summary>
        public const float WageBeatPeriodSec = 1f;

        /// <summary>
        /// Beat carry seeded on assignment so the first wage beat lands at half the
        /// beat period: inside the 1.0s red-line window with margin even under frame
        /// jitter (pack section 9 anchor: first dispatch feedback <= 1.0s).
        /// </summary>
        public const float FirstBeatCarrySeedSec = WageBeatPeriodSec * 0.5f;

        // ----- T005 pack section 6 mirrors (offline earnings law) -----
        /// <summary>Offline production = online efficiency x 0.6 (pack: offline yield is 60% of online).</summary>
        public const double OfflineCoef = 0.6;
        /// <summary>Offline wide cap = 12h (real-idle anchor; far looser than the genre's harsh 180min caps).</summary>
        public const double OfflineCapSec = 12.0 * 3600.0;
        /// <summary>Return-visit double offer multiplier, once per day (pack section 7; R73 red line: every ad gain <= x2).
    </summary>
        public const double OfflineDoubleMult = 2.0;

        // ----- T006 pack section 5 mirrors (comedy event engine) -----
        /// <summary>Scripted first event card beat: inside the <=10s red-line anchor with margin (pack section 9).</summary>
        public const float FirstEventCardAtSec = 8f;
    }
}

```

```

/// <summary>Online event cadence window low bound (pack section 5: one event every 180-300s).</summary>
public const double EventCadenceMinSec = 180.0;
/// <summary>Online event cadence window high bound.</summary>
public const double EventCadenceMaxSec = 300.0;

// ----- T007 pack section 7 mirrors (IAA triple: optional, x2-capped) -----
/// <summary>Ad gain red line: every ad boost multiplies wages by exactly x2
/// (pack section 7 law 1, R73 evidence "20x drops = ugly"). Boosts never
/// stack beyond it - rescue and speed windows cap at x2 together.</summary>
public const double AdBoostAmp = 2.0;
/// <summary>Rescue / speed boost window: 30 minutes (pack section 7 table).</summary>
public const double AdBoostDurSec = 1800.0;

// ----- live state -----
double _simTime;
double _coin;
double _starDust;
int _starLevel;
int _rankIndex;
int _dexCount; // recruited pet count (determines the star dust dex bonus)
readonly bool[] _petRecruited = new bool[PetData.PetCount];
readonly bool[] _jobUnlocked = new bool[PetData.JobCount];
readonly int[] _jobLevel = new int[PetData.JobCount];
readonly int[] _jobPet = new int[PetData.JobCount]; // occupant pet id, -1 = empty
readonly double[] _jobBeatCarry = new double[PetData.JobCount];
bool _hasDoubleOfferPending; // T005 return-visit double offer slot (one claim per settle)

// ----- T006 event engine state (pack section 5) -----
bool _firstEventFired; // scripted E01 boot beat fired
int _eventResolveCount; // resolved event ordinal (cadence roll seed)
double _nextEventAt = -1.0; // sim-time of the next cadence beat; -1 = none scheduled
readonly bool[] _eventCollected = new bool[PetData.EventCount]; // event album bits (SYS08)

// ----- T007 IAA boost state (pack section 7; data face - the View owns offers) -----
double _rescueBoostUntil = -1.0; // sim-time; shop-wide x2 wage window
int _speedBoostJob = -1; // -1 = none
double _speedBoostUntil = -1.0; // single-job x2 wage window

// ----- T008 postcard state (pack section 8; data face - slice B renders cards) -----
readonly double[] _pairWorkSec = new double[PetData.PostcardPairCount]; // work-time progress bars (never decay)
readonly bool[] _pairCardCollected = new bool[PetData.PostcardPairCount]; // album collect bits
readonly List<PostcardRecord> _postcards = new List<PostcardRecord>(16); // bounded journal (capacity law)
int _postcardsSent; // journal ordinals (wage grants + away summary cards)

// ----- T012 achievement state (SYS08 honor wall; data face - the View renders) -----
readonly bool[] _achievementUnlocked = new bool[PetData.AchievementCount];
int _adsWatched; // rewarded views consumed on granted boosts/claims (zero-ad honor condition)
int _lastAchievementId = -1; // latest unlock ordinal (-1 = none yet)

// ----- T013 endgame state (M10 finale; data face - the View renders the ceremony) -----

```

```

bool _endgameFired;           // one-shot ready event latch (fires once per run)
bool _endgameShown;          // the finale played (one ending per save file - M10 law)

// ----- T013 slice C: NG+ inheritance state (M10 second run; data face) -----
int _ngPlusCount;             // completed restarts that carried over (0 = first run)

// ----- red line 11 timing capture -----
public float HookReadyT { get; private set; } = -1f;
public float FirstDispatchT { get; private set; } = -1f;
public float FirstWageBeatT { get; private set; } = -1f;
public float FirstRecruitT { get; private set; } = -1f;
public float FirstOpenJobT { get; private set; } = -1f;
public float FirstStarUpT { get; private set; } = -1f;
public float FirstPromoteT { get; private set; } = -1f;
public float FirstEventT { get; private set; } = -1f; // T006: first event card (E01 first-laugh beat)
public float FirstAchievementT { get; private set; } = -1f; // T012: first honor unlock (honor wall beat)
public int WageBeatCount { get; private set; }

/// <summary>First dispatch to first wage beat gap, -1 before the pair completes.</summary>
public float DispatchToFirstBeatSec
    => (FirstDispatchT >= 0f && FirstWageBeatT >= 0f) ? FirstWageBeatT - FirstDispatchT : -1f;

// ----- sim events (editor gates subscribe for timestamp-capture assertions) -----
public event Action<float> OnHookReady; // t
public event Action<int, int, float> OnDispatched; // jobId, petId, t
public event Action<float, double, int> OnCoinBeat; // t, delta, jobId
public event Action<int, float> OnPetRecruited; // petId, t
public event Action<int, float> OnPetGifted; // petId, t (milestone grant channel, zero coin)
public event Action<int, float> OnJobOpened; // jobId, t
public event Action<int, int, float> OnJobUpgraded; // jobId, newLevel, t
public event Action<int, float> OnStarUp; // starLevel, t
public event Action<int, float> OnPromoted; // newRank, t
public event Action<OfflineSettleResult, float> OnOfflineSettled; // settle record, t (T005)
public event Action<double, float> OnOfflineDoubleClaimed; // bonus coin, t (T005)
public event Action<int, string, double, float> OnEventResolved; // eventId, fx, starDelta, t (T006)
public event Action<int, int, float> OnAdBoostActivated; // kind, jobId, t (T007; -1 = shop-wide)
public event Action<PostcardRecord, float> OnPostcardSent; // record, t (T008; wage grant or away summary)
public event Action<int, float> OnAchievementUnlocked; // achId, t (T012; honor wall beat)
public event Action<float> OnEndgameReady; // t (T013; full-rank all-lit finale, once per file)
public event Action<int, float> OnNewGamePlusStarted; // ngPlusCount, t (T013 slice C; the restart carried)

public PetSim()
{
    for (int i = 0; i < PetData.JobCount; i++) _jobPet[i] = -1;
}

// ----- readouts -----
public double SimTime => _simTime;
public double Coin => _coin;
public double StarDust => _starDust;

```

```

public int StarLevel => _starLevel;
public int RankIndex => _rankIndex;
public int DexCount => _dexCount;
public bool IsPetRecruited(int petId) => _petRecruited[petId];

// ----- T003 pet album (SYS03/SYS08 data face): one dex page per pet, unlocks on recruit/gift -----
public bool PetAlbumUnlocked(int petId)
    => petId >= 0 && petId < PetData.PetCount && _petRecruited[petId];
public int PetAlbumOpenCount => _dexCount;
public bool PetAlbumFull => _dexCount == PetData.PetCount;
public bool IsJobUnlocked(int jobId) => _jobUnlocked[jobId];
public int JobLevel(int jobId) => _jobLevel[jobId];
public int JobOccupant(int jobId) => _jobPet[jobId];
public int JobsOpenCount { get; private set; }

// ----- T005 offline settle readouts (return-visit summary data face) -----
/// <summary>Last settle record; the View slice renders the return-visit receipt from it.</summary>
public OfflineSettleResult LastOfflineSettle { get; private set; }
/// <summary>True when the last settle raised a claimable x2 offer slot.</summary>
public bool HasDoubleOfferPending => _hasDoubleOfferPending;

// ----- T006 event engine readouts (data face; slice B renders the card pop layer) -----
/// <summary>Resolved event count (cadence ordinal).</summary>
public int EventResolveCount => _eventResolveCount;
/// <summary>Event album (SYS08) collect bit per event id.</summary>
public bool IsEventCollected(int eventId)
    => eventId >= 0 && eventId < PetData.EventCount && _eventCollected[eventId];
/// <summary>Collected event album entries.</summary>
public int EventAlbumOpenCount
{
    get { int c = 0; for (int e = 0; e < PetData.EventCount; e++) if (_eventCollected[e]) c++; return c; }
}
/// <summary>True when the whole event pool is in the album (SYS08 full face).</summary>
public bool EventAlbumFull => EventAlbumOpenCount == PetData.EventCount;

// ----- T007 IAA boost readouts (data face; the View/slice B renders offers) -----
/// <summary>Rescue boost kind id for the OnAdBoostActivated event.</summary>
public const int BoostKindRescue = 0;
/// <summary>Speed boost kind id for the OnAdBoostActivated event.</summary>
public const int BoostKindSpeed = 1;
/// <summary>True while the shop-wide rescue x2 wage window is open.</summary>
public bool RescueBoostActive => _simTime < _rescueBoostUntil;
/// <summary>Rescue window remaining seconds (0 when closed).</summary>
public double RescueBoostRemainSec => RescueBoostActive ? _rescueBoostUntil - _simTime : 0.0;
/// <summary>Speed-boosted job id, -1 when no window is open.</summary>
public int SpeedBoostJob => _speedBoostUntil > _simTime ? _speedBoostJob : -1;
/// <summary>True while the given job's speed x2 window is open.</summary>
public bool SpeedBoostActive(int jobId) => _speedBoostUntil > _simTime && _speedBoostJob == jobId;

// ----- T008 postcard readouts (data face; slice B renders album + cards) -----

```

```

/// <summary>Total postcards sent (wage grants + away cards).</summary>
public int PostcardsSentTotal => _postcardsSent;
/// <summary>Bounded journal size (pack capacity law).</summary>
public int PostcardJournalCount => _postcards.Count;
/// <summary>Journal entry accessor for gates and the View feed.</summary>
public PostcardRecord PostcardAt(int index) => _postcards[index];
/// <summary>Most recent journal entry, null when empty.</summary>
public PostcardRecord LastPostcard => _postcards.Count > 0 ? _postcards[_postcards.Count - 1] : null;
/// <summary>Work-time progress bar of one (pet, job) pair, in seconds (never decays).</summary>
public double PairPostcardProgressSec(int petId, int jobId)
    => (petId >= 0 && petId < PetData.PetCount && jobId >= 0 && jobId < PetData.JobCount)
        ? _pairWorkSec[petId * PetData.JobCount + jobId] : 0.0;
/// <summary>Album collect bit of one (pet, job) pair card.</summary>
public bool IsPairPostcardCollected(int petId, int jobId)
    => (petId >= 0 && petId < PetData.PetCount && jobId >= 0 && jobId < PetData.JobCount)
        && _pairCardCollected[petId * PetData.JobCount + jobId];
/// <summary>Collected pair cards (album open pages).</summary>
public int PairPostcardOpenCount
{
    get { int c = 0; for (int k = 0; k < PetData.PostcardPairCount; k++) if (_pairCardCollected[k]) c++; return c; }
}
/// <summary>True when the whole pair album is collected (SYS08 full face).</summary>
public bool PairPostcardAlbumFull => PairPostcardOpenCount == PetData.PostcardPairCount;

// ----- T012 achievement readouts (data face; the View renders the honor wall) -----
/// <summary>Honor unlock bit per achievement id.</summary>
public bool IsAchievementUnlocked(int achId)
    => achId >= 0 && achId < PetData.AchievementCount && _achievementUnlocked[achId];
/// <summary>Unlocked honor count (honor wall progress face).</summary>
public int AchievementUnlockedCount
{
    get { int c = 0; for (int a = 0; a < PetData.AchievementCount; a++) if (_achievementUnlocked[a]) c++; return c; }
}
/// <summary>True when the whole honor wall is gold (legend face).</summary>
public bool AllAchievementsUnlocked => AchievementUnlockedCount == PetData.AchievementCount;
/// <summary>Rewarded ad views consumed on granted boosts/claims (U091 zero-ad honor condition face).</summary>
public int AdsWatchedCount => _adsWatched;
/// <summary>Latest unlock ordinal, -1 before the first honor.</summary>
public int LastAchievementId => _lastAchievementId;

// ----- T013 endgame readouts (M10: full-rank + every post lit finale) -----
/// <summary>Finale condition: top rank reached AND all jobs open (all posts lit).</summary>
public bool EndgameReady => _rankIndex >= PetData.RankCount - 1 && JobsOpenCount >= PetData.JobCount;
/// <summary>The finale ceremony already played (one ending per save file - M10 law).</summary>
public bool EndgameShown => _endgameShown;
/// <summary>One-way latch the View flips when it opens the finale (mark on open:
/// a mid-ceremony quit never replays the ending). Idempotent by design.</summary>
public void MarkEndgameShown()
{
    if (_endgameShown) return;

```

```

    _endgameShown = true;
}

// ----- T013 slice C: NG+ inheritance (M10 second run; data face - the
// View slice owns the entry button and the carried-over celebration) -----

/// <summary>Completed restarts that carried over (0 = first run).</summary>
public int NewGamePlusCount => _ngPlusCount;

public enum NgPlusResult { Ok = 0, NotReady = 1 }

/// <summary>
/// T013 slice C: start the second run. Eligibility = the finale condition
/// (top rank AND every post lit - pack section 11 freeze). Carried faces
/// ride the section 11 coefficients (both machine-set 1.0 = full carry):
/// pet dex album, event album, postcard pair album, the bounded journal,
/// the honor wall and the ads counter - a restart never wipes a
/// collection page (R72 / R73 anchors). Everything economic resets to a
/// fresh boot: coin, star dust, star level, rank ladder, job posts,
/// dispatch seats, boost windows, the offline offer slot, pair work bars,
/// the red-line first-anchors (the new run replays its own hook / first
/// dispatch / E01 first-laugh beats) and the finale latches - a new run
/// is a new ending slot (M10 one-ending-per-file law). Every refusal is
/// a zero-state-change verdict. Deterministic: the restart is a pure
/// function of the pre-state, so twin runs stay byte-identical across it.
/// </summary>
public NgPlusResult TryStartNewGamePlus()
{
    if (!EndgameReady) return NgPlusResult.NotReady; // refused, zero state change

    // economic reset (fresh boot faces)
    _simTime = 0.0;
    _coin = 0.0;
    _starDust = 0.0;
    _starLevel = 0;
    _rankIndex = 0;
    for (int j = 0; j < PetData.JobCount; j++)
    {
        _jobUnlocked[j] = false;
        _jobLevel[j] = 0;
        _jobPet[j] = -1;
        _jobBeatCarry[j] = 0.0;
    }
    _jobsOpenCount = 0;
    _hasDoubleOfferPending = false;
    _lastOfflineSettle = null;
    _rescueBoostUntil = -1.0;
    _speedBoostJob = -1;
    _speedBoostUntil = -1.0;
    _firstEventFired = false;

```

```

    _eventResolveCount = 0;
    _nextEventAt = -1.0;
    LastEventCard = null;
    for (int k = 0; k < PetData.PostcardPairCount; k++) _pairWorkSec[k] = 0.0;
    _endgameFired = false;
    _endgameShown = false;
    HookReadyT = -1f;
    FirstDispatchT = -1f;
    FirstWageBeatT = -1f;
    FirstRecruitT = -1f;
    FirstOpenJobT = -1f;
    FirstStarUpT = -1f;
    FirstPromoteT = -1f;
    FirstEventT = -1f;
    FirstAchievementT = -1f;
    WageBeatCount = 0;

    // fresh-boot seed: the free coffee post re-opens (the starter cat is
    // already on the dex from the carry - the recruit folds to a silent
    // no-op by the carried bits, so the seed inlines the post open only)
    _jobUnlocked[PetData.JobCoffee] = true;
    _jobLevel[PetData.JobCoffee] = 1;
    _jobPet[PetData.JobCoffee] = -1;
    JobsOpenCount = 1;

    _ngPlusCount++;
    OnNewGamePlusStarted?.Invoke(_ngPlusCount, (float)_simTime);
    return NgPlusResult.Ok;
}

/// <summary>Event card data face the View pop layer (slice B) renders.</summary>
public sealed class EventCard
{
    public int EventId;
    public string Id;           // localization key face (event_e01..)
    public string Fx;          // view fx stem
    public double StarDelta;    // star dust settled by this occurrence
}

/// <summary>Last resolved card; the View slice shows one modal per resolve.</summary>
public EventCard LastEventCard { get; private set; }

/// <summary>Occupied job count (derived, drift-proof across reassign/displace ops).</summary>
public int OnDutyPetCount
{
    get
    {
        int c = 0;
        for (int j = 0; j < PetData.JobCount; j++)
            if (_jobUnlocked[j] && _jobPet[j] >= 0) c++;
        return c;
    }
}

```

```

    }
}

// ----- op results (every refusal is a zero-state-change verdict) -----
public enum AssignResult { Ok = 0, PetLocked = 1, JobLocked = 2 }
public enum RecruitResult { Ok = 0, AlreadyRecruited = 1, NoCoin = 2 }
public enum OpenJobResult { Ok = 0, AlreadyOpen = 1, RankLocked = 2, NoCoin = 3 }
public enum UpgradeResult { Ok = 0, JobLocked = 1, NoCoin = 2 }
public enum PromoteResult { Ok = 0, MaxRank = 1, GatesNotMet = 2 }
public enum BoostResult { Ok = 0, AdNotConsumed = 1, JobLocked = 2, JobEmpty = 3, AlreadyActive = 4 }

/// <summary>Boot seed: starter cat gift + free first job open (pack section 1/4).</summary>
public void SeedStarterState()
{
    RecruitPet(PetData.PetOrange);
    OpenJob(PetData.JobCoffee);
}

public RecruitResult RecruitPet(int petId)
{
    if (petId < 0 || petId >= PetData.PetCount) return RecruitResult.NoCoin; // invalid id = refused, zero change
    if (_petRecruited[petId]) return RecruitResult.AlreadyRecruited;
    double cost = PetData.RecruitCosts[petId];
    if (_coin < cost) return RecruitResult.NoCoin;
    _coin -= cost;
    _petRecruited[petId] = true;
    _dexCount++;
    if (FirstRecruitT < 0f) FirstRecruitT = (float)_simTime;
    OnPetRecruited?.Invoke(petId, (float)_simTime);
    EvaluateAchievements();
    return RecruitResult.Ok;
}

/// <summary>
/// T003: milestone gift channel (pack section 4 - recruit = coin direct-buy +
/// milestone grants, zero gacha, zero probability). Grants a pet without
/// touching the coin balance; every refusal is a zero-state-change verdict.
/// Invalid ids fold into the NoCoin verdict (refusal family, zero change).
/// Fire-station companion wiring lands with the T004 job-pool slice which
/// owns OpenJob.
/// </summary>
public RecruitResult GiftPet(int petId)
{
    if (petId < 0 || petId >= PetData.PetCount) return RecruitResult.NoCoin; // refused, zero change
    if (_petRecruited[petId]) return RecruitResult.AlreadyRecruited;
    _petRecruited[petId] = true;
    _dexCount++;
    if (FirstRecruitT < 0f) FirstRecruitT = (float)_simTime;
    OnPetGifted?.Invoke(petId, (float)_simTime);
    EvaluateAchievements();
}

```

```

        return RecruitResult.Ok;
    }

    public OpenJobResult OpenJob(int jobId)
    {
        if (jobId < 0 || jobId >= PetData.JobCount) return OpenJobResult.RankLocked; // invalid id = refused
        if (!_jobUnlocked[jobId]) return OpenJobResult.AlreadyOpen;
        if (_rankIndex < PetData.Jobs[jobId].UnlockRank) return OpenJobResult.RankLocked;
        double cost = PetData.Jobs[jobId].OpenCost;
        if (_coin < cost) return OpenJobResult.NoCoin;
        _coin -= cost;
        _jobUnlocked[jobId] = true;
        _jobLevel[jobId] = 1;
        _jobPet[jobId] = -1;
        JobsOpenCount++;
        // Fire-station companion wiring (pack section 4: Awang is the fire-station
        // end-game companion pet): opening the terminal easter-egg job grants him
        // through the milestone gift channel - zero coin, and an already-recruited
        // Awang folds to a silent no-op so the egg never blocks or punishes.
        if (jobId == PetData.JobFire && !_petRecruited[PetData.PetAwang])
            GiftPet(PetData.PetAwang);
        if (FirstOpenJobT < 0f) FirstOpenJobT = (float)_simTime;
        OnJobOpened?.Invoke(jobId, (float)_simTime);
        EvaluateAchievements();
        return OpenJobResult.Ok;
    }

    /// <summary>
    /// Dispatch op: move a recruited pet onto an unlocked job. Reassignment is free
    /// of punishment (pack law: flipping a pet is a comedy beat, never a penalty):
    /// the pet leaves its old job empty, the displaced occupant (if any) goes idle.
    /// </summary>
    public AssignResult AssignPet(int jobId, int petId)
    {
        if (jobId < 0 || jobId >= PetData.JobCount || petId < 0 || petId >= PetData.PetCount)
            return AssignResult.JobLocked; // invalid id = refused, zero change
        if (!_petRecruited[petId]) return AssignResult.PetLocked;
        if (!_jobUnlocked[jobId]) return AssignResult.JobLocked;

        for (int j = 0; j < PetData.JobCount; j++)
        {
            if (_jobPet[j] == petId)
            {
                if (j == jobId) return AssignResult.Ok; // no-op re-dispatch
                _jobPet[j] = -1;
                _jobBeatCarry[j] = 0.0f;
            }
        }
        // a displaced old occupant (if any) simply goes idle - comedy beat, never a penalty
        _jobPet[jobId] = petId;
    }

```

```

    _jobBeatCarry[jobId] = FirstBeatCarrySeedSec; // first beat inside the 1.0s window
    if (FirstDispatchT < 0f) FirstDispatchT = (float)_simTime;
    OnDispatched?.Invoke(jobId, petId, (float)_simTime);
    return AssignResult.Ok;
}

public UpgradeResult UpgradeJob(int jobId)
{
    if (jobId < 0 || jobId >= PetData.JobCount || !_jobUnlocked[jobId]) return UpgradeResult.JobLocked;
    double cost = PetData.JobUpgradeCost(jobId, _jobLevel[jobId]);
    if (_coin < cost) return UpgradeResult.NoCoin;
    _coin -= cost;
    _jobLevel[jobId]++;
    OnJobUpgraded?.Invoke(jobId, _jobLevel[jobId], (float)_simTime);
    return UpgradeResult.Ok;
}

/// <summary>Promote gate: monotonic triple threshold (jobs / dex / stars), zero lockout face.</summary>
public PromoteResult TryPromote()
{
    if (_rankIndex >= PetData.RankCount - 1) return PromoteResult.MaxRank;
    int g = _rankIndex; // gate from rank g to g+1
    if (JobsOpenCount < PetData.GateJobsAtLeast[g] ||
        _dexCount < PetData.GateDexAtLeast[g] ||
        _starLevel < PetData.GateStarsAtLeast[g])
        return PromoteResult.GatesNotMet;
    _rankIndex++;
    if (FirstPromoteT < 0f) FirstPromoteT = (float)_simTime;
    OnPromoted?.Invoke(_rankIndex, (float)_simTime);
    EvaluateAchievements();
    return PromoteResult.Ok;
}

// ----- T005 offline settlement (pack section 6: offline = online x 0.6, 12h wide cap) -----

/// <summary>Offline settle record (return-visit summary data face).</summary>
public sealed class OfflineSettleResult
{
    public double AwaySec; // raw away span fed by the caller (real-clock face)
    public double CreditedSec; // min(away, 12h cap)
    public double OnlineRatePerSec; // online efficiency snapshot at settle time
    public double Coin; // credited wage coin (before the optional double)
    public double StarDust; // credited star dust
    public int OnDuty; // pets on duty at settle
}

/// <summary>
/// T005 offline settle op (data face). Offline production = online efficiency
/// x 0.6 with the 12h wide cap; both resource streams (wages + star dust)
/// settle together because the pack law prices the whole online efficiency,

```

```

/// not just the coin face. The lump credit is deterministic (a pure function
/// of the away span + the live roster), leaves the dispatch state untouched,
/// and raises one return-visit double-offer slot when there is something to
/// double (empty roster = honest zero, no hostage UX). Star-ups feed on the
/// credited star dust on the NEXT online tick (the Tick while-loop owns level
/// transitions - settling never mutates rank/dex/dispatch state). Refusals
/// (away &lt;= 0) are zero-state-change verdicts.
/// </summary>
public OfflineSettleResult SettleOffline(double awaySec)
{
    if (awaySec <= 0.0)
    {
        var empty = new OfflineSettleResult { AwaySec = awaySec, CreditedSec = 0.0 };
        OnOfflineSettled?.Invoke(empty, (float)_simTime);
        return empty; // refused, zero state change
    }

    double rate = 0.0;
    for (int j = 0; j < PetData.JobCount; j++)
    {
        if (_jobUnlocked[j] && _jobPet[j] >= 0)
            rate += PetData.JobRatePerSec(j, _jobLevel[j]);
    }
    double creditedSec = Math.Min(awaySec, OfflineCapSec);
    int onDuty = OnDutyPetCount;
    double starPerMin = PetData.StarDustPerMinPerPet * onDuty
        * (1.0 + PetData.StarDustDexBonusCoef * _dexCount);
    var r = new OfflineSettleResult
    {
        AwaySec = awaySec,
        CreditedSec = creditedSec,
        OnlineRatePerSec = rate,
        OnDuty = onDuty,
        Coin = rate * creditedSec * OfflineCoef,
        StarDust = starPerMin / 60.0 * creditedSec * OfflineCoef,
    };
    _coin += r.Coin;
    _starDust += r.StarDust;
    _hasDoubleOfferPending = r.Coin > 0.0; // nothing to double = no offer face
    // T008: away work credits the on-duty pairs' postcard bars (true idle
    // game - pack section 8 bars accumulate across away spans), grants any
    // bars that fill, then journals the return-visit summary postcard
    // (requirement doc M08 "the crew writes home"; zero summary = no card,
    // no hostage UX). Grants are pure records - coin / star faces stay exact.
    for (int j = 0; j < PetData.JobCount; j++)
    {
        int occ = _jobPet[j];
        if (occ < 0 || !_jobUnlocked[j]) continue;
        int k = occ * PetData.JobCount + j;
        _pairWorkSec[k] += creditedSec;
    }
}

```

```

        TryGrantPairPostcard(occ, j);
    }
    if (r.Coin > 0.0)
        AppendPostcard(new PostcardRecord
        {
            Seq = ++_postcardsSent,
            Kind = PetData.PostcardKind.Away,
            Pet = -1,
            Job = -1,
            WorkSec = creditedSec,
            Coin = r.Coin,
            StarDust = r.StarDust,
        });
    LastOfflineSettle = r;
    OnOfflineSettled?.Invoke(r, (float)_simTime);
    EvaluateAchievements();
    return r;
}

/// <summary>
/// T005 return-visit double offer claim (pack section 6/7: 1/day, x2; R73 red
/// line: every ad gain &lt;= x2). Data-layer semantics: one settle raises at most
/// one claimable slot; claiming credits the base amount once more (base + base
/// = x2 total) and consumes the slot - the base is never held hostage (U091:
/// experience first, the ad is an optional bonus). T007 adds the ad gate: the
/// caller must certify the rewarded view was consumed (AdCap g15_double
/// pre-check + TryConsumeRewarded live in the SYS10 caller face); false =
/// zero-state-change refusal and the slot stays claimable. The per-day
/// frequency budget belongs to the AdCap category policy (T007 AdCapBoostCheck
/// owns HasCategoryBudget); this op owns the slot state machine only.
/// </summary>
public bool TryClaimOfflineDouble(bool adConsumed = true)
{
    if (!adConsumed) return false; // T007 ad gate: refused, zero change
    if (!_hasDoubleOfferPending || LastOfflineSettle == null) return false; // refused, zero change
    _hasDoubleOfferPending = false;
    double bonus = LastOfflineSettle.Coin * (OfflineDoubleMult - 1.0);
    _coin += bonus;
    _adsWatched++; // T012: the ad certificate was honored - the zero-ad honor counts it
    OnOfflineDoubleClaimed?.Invoke(bonus, (float)_simTime);
    EvaluateAchievements();
    return true;
}

// ----- T007 IAA boosts (pack section 7; data layer - offers are a View face) -----

/// <summary>
/// T007 rescue boost: the peak short-staff rescue ad opens a 30-minute
/// shop-wide x2 wage window (P02-T006 rescue paradigm). The AdCap budget
/// pre-check and consume live in the caller (SYS10 HasCategoryBudget /

```

```

/// TryConsumeRewarded on g15_rescue); this op grants only when the caller
/// certifies the view was consumed - false = zero-state-change refusal.
/// A window already open refuses (the View pre-offer gate hides the seat;
/// the data layer stays honest for direct calls). Zero-RNG law intact.
/// </summary>
public BoostResult TryActivateRescueBoost(bool adConsumed)
{
    if (!adConsumed) return BoostResult.AdNotConsumed;
    if (RescueBoostActive) return BoostResult.AlreadyActive;
    _rescueBoostUntil = _simTime + AdBoostDurSec;
    _adsWatched++; // T012: granted on a consumed rewarded view - zero-ad honor counts it
    OnAdBoostActivated?.Invoke(BoostKindRescue, -1, (float)_simTime);
    EvaluateAchievements();
    return BoostResult.Ok;
}

/// <summary>
/// T007 speed boost: the current-job ad opens a 30-minute x2 window on one
/// occupied job (pack section 7: "current job output x2"). Same caller-side
/// AdCap contract as the rescue seat (g15_speed); an empty or locked job
/// refuses with zero state change - the ad can never be wasted on a job
/// that produces nothing (U091 honesty law). Re-aiming a live window at a
/// different job refuses while the first window runs (single-seat law).
/// </summary>
public BoostResult TryActivateSpeedBoost(int jobId, bool adConsumed)
{
    if (!adConsumed) return BoostResult.AdNotConsumed;
    if (jobId < 0 || jobId >= PetData.JobCount || !_jobUnlocked[jobId]) return BoostResult.JobLocked;
    if (_jobPet[jobId] < 0) return BoostResult.JobEmpty;
    if (_speedBoostUntil > _simTime) return BoostResult.AlreadyActive;
    _speedBoostJob = jobId;
    _speedBoostUntil = _simTime + AdBoostDurSec;
    _adsWatched++; // T012: granted on a consumed rewarded view - zero-ad honor counts it
    OnAdBoostActivated?.Invoke(BoostKindSpeed, jobId, (float)_simTime);
    EvaluateAchievements();
    return BoostResult.Ok;
}

// ----- T006 event engine (pack section 5; data layer - slice A) -----

/// <summary>Deterministic 32-bit mix (zero RNG law: identical inputs, identical rolls).</summary>
static uint Mix32(uint x)
{
    x ^= x >> 16; x *= 0x7feb352du; x ^= x >> 15; x *= 0x846ca68bu; x ^= x >> 16;
    return x;
}

/// <summary>Cadence window roll: 180-300s slot picked deterministically per ordinal.</summary>
static double CadenceSecFor(int ordinal)
{

```

```

    double span = EventCadenceMaxSec - EventCadenceMinSec;
    uint slot = Mix32((uint)ordinal) % (uint)(span + 1.0);
    return EventCadenceMinSec + slot;
}

/// <summary>
/// Roll the next cadence event among events whose job has a pet on duty (pack
/// law: random on-duty pet). Deterministic hash keeps the zero-RNG isomorphism
/// law intact; empty eligible set folds to -1 (honest skip, no hostage face).
/// Peak-hour weighting is a real-clock View face and lands with slice B.
/// </summary>
int RollCadenceEvent()
{
    int eligible = 0;
    for (int e = 0; e < PetData.EventCount; e++)
    {
        int j = PetData.Events[e].Job;
        if (_jobUnlocked[j] && _jobPet[j] >= 0) eligible++;
    }
    if (eligible == 0) return -1;
    uint pick = Mix32(0x9e3779b9u + (uint)_eventResolveCount) % (uint)eligible;
    for (int e = 0; e < PetData.EventCount; e++)
    {
        int j = PetData.Events[e].Job;
        if (!(_jobUnlocked[j] && _jobPet[j] >= 0)) continue;
        if (pick == 0) return e;
        pick--;
    }
    return -1; // unreachable
}

/// <summary>
/// Settle one event occurrence: star-dust correction (pack window) + album
/// collect bit (SYS08) + card data face for the View pop layer. Star level-ups
/// feed on the next star pass in Tick (the pass owns level transitions).
/// </summary>
void ResolveEvent(int eventId)
{
    var def = PetData.Events[eventId];
    _starDust += def.StarDelta;
    if (def.Collect) _eventCollected[eventId] = true;
    _eventResolveCount++;
    LastEventCard = new EventCard { EventId = eventId, Id = def.Id, Fx = def.Fx, StarDelta = def.StarDelta };
    OnEventResolved?.Invoke(eventId, def.Fx, def.StarDelta, (float)_simTime);
}

/// <summary>
/// Advance the idle clock: hook beat on frame zero, star dust natural accumulation
/// (no check-in, no contest), per-job wage beats on the 1s cadence while occupied.
/// </summary>

```

```

public void Tick(float dt)
{
    if (dt <= 0f) return;
    if (HookReadyT < 0f)
    {
        HookReadyT = (float)_simTime; // hook content lives from frame zero
        OnHookReady?.Invoke((float)_simTime);
    }
    _simTime += dt;

    if (OnDutyPetCount > 0)
    {
        double perMin = PetData.StarDustPerMinPerPet * OnDutyPetCount
            * (1.0 + PetData.StarDustDexBonusCoef * _dexCount);
        _starDust += perMin / 60.0 * dt;
        while (_starLevel < PetData.StarMax &&
            _starDust >= PetData.StarUpCostBase * (_starLevel + 1))
        {
            _starDust -= PetData.StarUpCostBase * (_starLevel + 1);
            _starLevel++;
            if (FirstStarUpT < 0f) FirstStarUpT = (float)_simTime;
            OnStarUp?.Invoke(_starLevel, (float)_simTime);
        }
    }

    bool rescueOn = RescueBoostActive; // T007: snapshot once per tick (window readouts)
    for (int j = 0; j < PetData.JobCount; j++)
    {
        if (!_jobUnlocked[j] || _jobPet[j] < 0) continue; // empty job produces nothing
        _jobBeatCarry[j] += dt;
        while (_jobBeatCarry[j] >= WageBeatPeriodSec)
        {
            _jobBeatCarry[j] -= WageBeatPeriodSec;
            // T007: rescue (shop-wide) and speed (single job) windows boost the
            // beat, but never stack beyond the x2 red line (pack section 7 law 1).
            bool boosted = rescueOn || SpeedBoostActive(j);
            double delta = PetData.JobRatePerSec(j, _jobLevel[j]) * WageBeatPeriodSec
                * (boosted ? AdBoostAmp : 1.0);
            _coin += delta;
            WageBeatCount++;
            // T008: attribute the beat to the occupying pet's (pet, job)
            // postcard progress bar (pack section 8 - work time accumulates).
            int occ = _jobPet[j];
            if (occ >= 0) _pairWorkSec[occ * PetData.JobCount + j] += WageBeatPeriodSec;
            if (FirstWageBeatT < 0f) FirstWageBeatT = (float)_simTime;
            OnCoinBeat?.Invoke((float)_simTime, delta, j);
        }
    }

    // T006: comedy event beats (pack section 5). First card = scripted E01

```

```

// boot beat inside the <=10s anchor (red line 11 first-laugh beat); later
// beats ride the 180-300s cadence window and roll among events whose
// job has a pet on duty. Empty eligible set = honest skip, rescheduled
// on the same window (zero-RNG law keeps fingerprints byte-identical).
if (!_firstEventFired && _simTime >= FirstEventCardAtSec)
{
    _firstEventFired = true;
    FirstEventT = (float)_simTime;
    ResolveEvent(0); // E01 corgi tumble: the scripted first-laugh card
    _nextEventAt = _simTime + CadenceSecFor(_eventResolveCount);
}
else if (_nextEventAt > 0.0 && _simTime >= _nextEventAt)
{
    int e = RollCadenceEvent();
    if (e >= 0) ResolveEvent(e);
    _nextEventAt = _simTime + CadenceSecFor(_eventResolveCount);
}

// T008: wage postcard pass (pack section 8): one collectible per (pet,
// job) pair per full 4h work bar. Progress-bar collection - bars
// accumulate per occupied beat and never decay (zero probability,
// guaranteed payout - R72 "letters were pure luck" counter-evidence).
for (int j = 0; j < PetData.JobCount; j++)
{
    int occ = _jobPet[j];
    if (occ >= 0 && _jobUnlocked[j]) TryGrantPairPostcard(occ, j);
}

// T012: honor pass - wage beats, star-ups, event resolves and postcard
// grants all land inside Tick, so the wall evaluates once per tick.
EvaluateAchievements();

// T013: finale latch (M10) - the ready condition can complete either on
// a promote or on the last job opening, so the check rides the tick: it
// fires OnEndgameReady exactly once per run, the frame the condition
// first holds. The shown bit stays the View's latch (one ending per file).
if (!_endgameFired && EndgameReady)
{
    _endgameFired = true;
    OnEndgameReady?.Invoke((float)_simTime);
}
}

// ----- T012 achievement engine (SYS08 honor wall; data layer - the View renders) -----

/// <summary>
/// Sequential honor evaluation: every not-yet-unlocked achievement checks
/// its condition against the live state; a met condition flips the unlock
/// bit and fires OnAchievementUnlocked the same pass. The legend (index 7)
/// evaluates last, so a single promote can cascade GlobalCorp -> ZeroAdClear

```

```

/// -> WorkLegend within one call. Conditions are one-way (an unlock never
/// re-locks) and every judgment maps 1:1 onto the PetData table comment.
/// </summary>
void EvaluateAchievements()
{
    for (int a = 0; a < PetData.AchievementCount; a++)
    {
        if (_achievementUnlocked[a]) continue;
        bool met;
        switch (a)
        {
            case PetData.AchFirstWage: met = WageBeatCount >= 1; break;
            case PetData.AchFullCrew: met = _dexCount >= PetData.PetCount; break;
            case PetData.AchShopStarFull: met = _starLevel >= PetData.StarMax; break;
            case PetData.AchPostcardCollector: met = _postcardsSent >= PetData.PostcardCollectorTarget; break;
            case PetData.AchEventWitness: met = EventAlbumOpenCount >= PetData.EventCount; break;
            case PetData.AchGlobalCorp: met = _rankIndex >= PetData.RankCount - 1; break;
            case PetData.AchZeroAdClear:
                met = _rankIndex >= PetData.RankCount - 1 && _adsWatched == 0;
                break;
            case PetData.AchWorkLegend:
                met = true;
                for (int o = 0; o < PetData.AchWorkLegend; o++)
                    if (!_achievementUnlocked[o]) { met = false; break; }
                break;
            default: met = false; break;
        }
        if (!met) continue;
        _achievementUnlocked[a] = true;
        _lastAchievementId = a;
        if (FirstAchievementT < 0f) FirstAchievementT = (float)_simTime;
        OnAchievementUnlocked?.Invoke(a, (float)_simTime);
    }
}

/// <summary>
/// Deterministic state fingerprint (offline/online isomorphism base): two runs
/// fed identical (dt, op) streams must produce identical strings, byte for byte.
/// </summary>
public string ComputeFingerprint()
{
    var sb = new StringBuilder(192);
    sb.Append(_simTime.ToString("R", CultureInfo.InvariantCulture)).Append('|');
    sb.Append(_coin.ToString("R", CultureInfo.InvariantCulture)).Append('|');
    sb.Append(_starDust.ToString("R", CultureInfo.InvariantCulture)).Append('|');
    sb.Append(_starLevel).Append('|').Append(_rankIndex).Append('|').Append(_dexCount).Append('|');
    for (int i = 0; i < PetData.PetCount; i++) sb.Append(_petRecruited[i] ? '1' : '0');
    sb.Append('|');
    for (int j = 0; j < PetData.JobCount; j++)
    {

```

```

        sb.Append(_jobUnlocked[j] ? '1' : '0').Append(':').Append(_jobLevel[j]).Append(':').Append(_jobPet[j]);
        if (j < PetData.JobCount - 1) sb.Append(';');
    }
    sb.Append('|').Append(_eventResolveCount).Append('|');
    for (int e = 0; e < PetData.EventCount; e++) sb.Append(_eventCollected[e] ? '1' : '0');
    sb.Append('|').Append(_rescueBoostUntil.ToString("R", CultureInfo.InvariantCulture))
        .Append('|').Append(_speedBoostJob)
        .Append('|').Append(_speedBoostUntil.ToString("R", CultureInfo.InvariantCulture))
        .Append('|').Append(_postcardsSent).Append('|').Append(_postcards.Count).Append('|');
    // T008 pair album bits, packed 4-per-hex-nibble (fixed-width face, 30 chars).
    for (int k = 0; k < PetData.PostcardPairCount; k += 4)
    {
        int nib = (_pairCardCollected[k] ? 8 : 0) | (_pairCardCollected[k + 1] ? 4 : 0)
            | (_pairCardCollected[k + 2] ? 2 : 0) | (_pairCardCollected[k + 3] ? 1 : 0);
        sb.Append(nib.ToString("X", CultureInfo.InvariantCulture));
    }
    // T012 honor wall faces: unlock bits + the zero-ad condition counter.
    sb.Append('|');
    for (int a = 0; a < PetData.AchievementCount; a++) sb.Append(_achievementUnlocked[a] ? '1' : '0');
    sb.Append('|').Append(_adsWatched);
    // T013 finale faces: ready latch + shown latch (one ending per file).
    sb.Append('|').Append(_endgameFired ? '1' : '0').Append(_endgameShown ? '1' : '0');
    // T013 slice C: NG+ restart ordinal (run counter across inheritance).
    sb.Append('|').Append(_ngPlusCount);
    return sb.ToString();
}

// ----- T008 postcard engine (pack section 8; data layer - slice B renders) -----

/// <summary>Postcard journal record: the share + album data face.</summary>
public sealed class PostcardRecord
{
    public int Seq;           // journal ordinal (1-based)
    public PetData.PostcardKind Kind; // Wage = pair collectible; Away = return-visit summary
    public int Pet;          // -1 on the away card
    public int Job;          // -1 on the away card
    public double WorkSec;   // credited work (wage) or away (away) seconds
    public double Coin;      // away card credited coin (wage cards are work-time collectibles)
    public double StarDust;  // away card credited star dust
}

/// <summary>Grant the (pet, job) pair card when its work bar fills. Pure
/// record: coin / star faces never move (a grant is a collection event,
/// not a payout - the wage itself already landed beat by beat).</summary>
void TryGrantPairPostcard(int petId, int jobId)
{
    int k = petId * PetData.JobCount + jobId;
    if (_pairCardCollected[k] || _pairWorkSec[k] < PetData.PostcardWorkSecPerCard) return;
    _pairCardCollected[k] = true;
    AppendPostcard(new PostcardRecord

```

```

    {
        Seq = ++_postcardsSent,
        Kind = PetData.PostcardKind.Wage,
        Pet = petId,
        Job = jobId,
        WorkSec = _pairWorkSec[k],
    });
}

/// <summary>Journal append with the pack capacity law (FIFO eviction at
/// cap). Static + public so the gate can prove the bound without 120
/// live grants; the sim uses it through the instance wrapper below.</summary>
public static void AppendPostcardBounded(List<PostcardRecord> journal, PostcardRecord rec, int capacity)
{
    if (capacity > 0 && journal.Count >= capacity) journal.RemoveAt(0);
    journal.Add(rec);
}

void AppendPostcard(PostcardRecord rec)
{
    AppendPostcardBounded(_postcards, rec, PetData.PostcardAlbumCapacity);
    OnPostcardSent?.Invoke(rec, (float)_simTime);
}

// ----- T008 share dual channel (red line 12 / X032 law: offline share
// form - a compact code string the receiver imports and re-derives
// locally, zero server, zero networking). Channel A (the rendered card
// image) is a View face and lands with slice B; this is channel B, the
// data face both channels feed from. Cent resolution on the away money
// fields keeps the roundtrip byte-exact (the code carries the summary
// face, not the raw doubles). -----

const string ShareCodePrefix = "G15PC1"; // game tag + code schema version

static string ToB36(long v)
{
    if (v == 0) return "0";
    bool neg = v < 0;
    if (neg) v = -v;
    const string alphabet = "0123456789ABCDEFGHIJKLMNPOQRSTUVWXYZ";
    var sb = new StringBuilder(12);
    while (v > 0) { sb.Append(alphabet[(int)(v % 36)]); v /= 36; }
    if (neg) sb.Append('-');
    char[] c = sb.ToString().ToCharArray();
    Array.Reverse(c);
    return new string(c);
}

static bool TryFromB36(string s, out long v)
{

```

```

    v = 0;
    if (string.IsNullOrEmpty(s)) return false;
    foreach (char ch in s)
    {
        int d = ch >= '0' && ch <= '9' ? ch - '0'
            : ch >= 'A' && ch <= 'Z' ? ch - 'A' + 10 : -1;
        if (d < 0) return false;
        v = v * 36 + d;
    }
    return true;
}

static uint Fnv1a(string s)
{
    uint h = 2166136261u;
    for (int i = 0; i < s.Length; i++) { h ^= (uint)s[i]; h *= 16777619u; }
    return h;
}

/// <summary>Channel B encoder: wage/away card -> compact offline share code.</summary>
public static string EncodePostcardCode(PostcardRecord rec)
{
    long coinCents = (long)Math.Round(rec.Coin * 100.0, MidpointRounding.AwayFromZero);
    long starCents = (long)Math.Round(rec.StarDust * 100.0, MidpointRounding.AwayFromZero);
    long workSec = (long)Math.Round(rec.WorkSec, MidpointRounding.AwayFromZero);
    string payload = ShareCodePrefix + "."
        + (rec.Kind == PetData.PostcardKind.Wage ? 'W' : 'A') + "."
        + ToB36(rec.Seq) + "." + ToB36(rec.Pet + 1) + "." + ToB36(rec.Job + 1) + "."
        + ToB36(workSec) + "." + ToB36(coinCents) + "." + ToB36(starCents);
    return payload + "." + ToB36((long)Fnv1a(payload));
}

/// <summary>Channel B decoder: validates the checksum, then re-derives the
/// card summary locally. Every refusal returns false with a null record
/// (tampered / malformed codes never touch sim state).</summary>
public static bool TryParsePostcardCode(string code, out PostcardRecord rec)
{
    rec = null;
    if (string.IsNullOrEmpty(code)) return false;
    int last = code.LastIndexOf('.');
    if (last <= 0 || last == code.Length - 1) return false;
    string payload = code.Substring(0, last);
    string check = code.Substring(last + 1);
    long want;
    if (!TryFromB36(check, out want) || want != (long)Fnv1a(payload)) return false;
    string[] f = payload.Split('.');
    if (f.Length != 8 || f[0] != ShareCodePrefix) return false;
    long seq, pet1, job1, workSec, coinCents, starCents;
    if (!TryFromB36(f[2], out seq) || !TryFromB36(f[3], out pet1) || !TryFromB36(f[4], out job1)
        || !TryFromB36(f[5], out workSec) || !TryFromB36(f[6], out coinCents) || !TryFromB36(f[7], out starCents))

```

```

        return false;
    if ((f[1] != "W" && f[1] != "A") || pet1 < 0 || pet1 > PetData.PetCount
        || job1 < 0 || job1 > PetData.JobCount || seq < 1)
        return false;
    rec = new PostcardRecord
    {
        Seq = (int)seq,
        Kind = f[1] == "W" ? PetData.PostcardKind.Wage : PetData.PostcardKind.Away,
        Pet = (int)(pet1 - 1),
        Job = (int)(job1 - 1),
        WorkSec = workSec,
        Coin = coinCents / 100.0,
        StarDust = starCents / 100.0,
    };
    return true;
}
}
}

```